🧠 AI Evolution Program

Parte 07 — IA generativa para texto, imagen, audio, video y 3D

Cubre la generación multimodal y sus pipelines creativos, incluyendo control, procedencia, consentimiento y evaluación de calidad.

1. 088 — Espacios latentes y autoencoders variacionales
2. 089 — GAN y entrenamiento adversarial
3. 090 — Modelos de difusión
4. 091 — Texto a imagen y condicionamiento
5. 092 — Control estructural y edición generativa
6. 093 — Generación musical y de audio
7. 094 — Síntesis de voz y derechos de identidad
8. 095 — Generación y edición de video
9. 096 — Generación 3D y mundos sintéticos
10. 097 — Datos sintéticos: utilidad y contaminación
11. 098 — Procedencia, marcas y autenticidad
12. 099 — Proyecto: pipeline creativo trazable

088 — Espacios latentes y autoencoders variacionales

Parte: 07 — IA generativa para texto, imagen, audio, video y 3D
Nivel: avanzado · **Horas estimadas:** 6
Laboratorio: `generation` · **Estado:** `EXECUTABLE_CORE`

🎯 Propósito

Comprender **espacios latentes y autoencoders variacionales** dentro de la evolución de la inteligencia artificial, implementar un experimento mínimo verificable y distinguir qué parte constituye evidencia frente a una afirmación todavía no comprobada.

📚 Resultados de aprendizaje

Al finalizar podrás:

1. Explicar espacios latentes y autoencoders variacionales usando los conceptos `VAE`, `latente`, `reconstrucción`, `muestreo`.
2. Ejecutar el laboratorio con una semilla explícita y revisar su contrato JSON.
3. Identificar al menos un supuesto, una limitación y un riesgo de aplicación.
4. Comparar el enfoque con la etapa anterior de la ruta de aprendizaje.
5. Producir una evidencia reproducible y una conclusión que no exceda los datos.

🧩 Conceptos centrales

`VAE`, `latente`, `reconstrucción`, `muestreo`

🗺️ Ubicación en el mapa de la IA

El autoencoder variacional (Kingma y Welling, 2013) inaugura la era moderna de los modelos generativos profundos: es el primer método que entrena una red neuronal como modelo probabilístico de variables latentes con gradiente de extremo a extremo. Hereda la inferencia variacional de la estadística bayesiana y el autoencoder clásico del aprendizaje de representaciones, y habilita lo que sigue en esta parte: los GAN (089) compiten con él en calidad de muestra, y los modelos de difusión (090) generalizan su cota variacional a una jerarquía de latentes. Los "espacios latentes" que hoy usan Stable Diffusion o los tokenizadores de imagen descienden directamente de esta idea.

📖 Fundamentos

🎲 Modelos de variables latentes

Un modelo de variable latente asume que cada dato observado x fue generado en dos pasos: se muestrea un vector latente z ~ p(z) (normalmente N(0, I)) y luego x ~ p_θ(x|z), donde p_θ(x|z) es una red neuronal (el **decoder**). La verosimilitud marginal es una integral intratable:

```
p_θ(x) = ∫ p_θ(x|z) · p(z) dz
```

Intratable porque habría que integrar sobre todos los z posibles. La inferencia inversa p_θ(z|x) (qué latente produjo este dato) también es intratable por Bayes.

🔧 Inferencia variacional amortizada y ELBO

El VAE introduce un **encoder** q_φ(z|x) — otra red neuronal que aproxima la posterior verdadera, típicamente una gaussiana diagonal N(μ_φ(x), diag(σ_φ²(x))). Con él se construye la **cota inferior de la evidencia** (ELBO):

```
log p_θ(x) ≥ ELBO(x) = E_{z~q_ϕ(z|x)}[ log p_θ(x|z) ] − KL( q_ϕ(z|x) ‖ p(z) )
```

La desigualdad es exacta: la brecha entre log p_θ(x) y el ELBO es exactamente KL(q_φ(z|x) ‖ p_θ(z|x)) ≥ 0. Maximizar el ELBO en θ y φ simultáneamente hace dos cosas: empuja la verosimilitud hacia arriba y ajusta q_φ hacia la posterior real.

Los dos términos tienen lectura directa:

- **Reconstrucción** E_q[log p_θ(x|z)]: el decoder debe reconstruir x desde un z muestreado del encoder. Con decoder gaussiano de varianza fija equivale a un error cuadrático; con decoder Bernoulli, a entropía cruzada.
- **Regularización** KL(q_φ(z|x) ‖ p(z)): castiga que el encoder se aleje del prior. Para dos gaussianas diagonales tiene forma cerrada por dimensión:

```
KL( N(μ_q, σ_q²) ‖ N(μ_p, σ_p²) ) = log(σ_p/σ_q) + (σ_q² + (μ_q − μ_p)²)/(2σ_p²) − 1/2
Con prior N(0,1):  KL = −log σ_q + (σ_q² + μ_q²)/2 − 1/2
```

🎯 Truco de reparametrización

El obstáculo: no se puede retropropagar a través de un muestreo z ~ N(μ, σ²), porque "muestrear" no es diferenciable respecto a μ y σ. La solución de Kingma y Welling es reescribir la aleatoriedad como entrada externa:

```
ε ~ N(0, I)          (ruido independiente de los parámetros)
z = μ_ϕ(x) + σ_ϕ(x) ⊙ ε    (transformación determinista y diferenciable)
```

Ahora z es una función diferenciable de φ y el gradiente ∂ELBO/∂φ fluye a través de μ y σ con un estimador de baja varianza. Sin este truco habría que usar REINFORCE, con varianza mucho mayor.

🗺️ El espacio latente como representación

Al entrenar, el VAE organiza los latentes: datos parecidos caen en regiones cercanas y el prior N(0, I) obliga a que el espacio esté "lleno" alrededor del origen — por eso interpolar entre dos z produce decodificaciones intermedias plausibles, algo que un autoencoder determinista no garantiza (su espacio latente puede tener "huecos" sin significado). El precio: la gaussiana diagonal de q_φ y el peso del término KL tienden a producir

reconstrucciones borrosas, y si el decoder es demasiado potente puede ignorar z (**posterior collapse**: KL → 0 y el latente no codifica nada).

🧮 Ejemplo trabajado

VAE de juguete en 1D: prior $p(z) = N(0, 1)$, encoder que para cierto x produce $\mu_q = 0.5$ y $\sigma_q = 0.8$, decoder gaussiano $p_\theta(x|z) = N(g(z), 1)$ con $g(z) = 2z$. Observación real: $x = 2.0$. Calculamos el ELBO con una sola muestra de ε.

Paso 1 — KL en forma cerrada (prior estándar: $\mu_p = 0$, $\sigma_p = 1$):

```
KL = −log σ_q + (σ_q² + μ_q²)/2 − 1/2
   = −log 0.8 + (0.64 + 0.25)/2 − 0.5
   = 0.2231 + 0.4450 − 0.5
   = 0.1681
```

Paso 2 — Reparametrización. Muestreamos $\varepsilon = 0.5$:

```
z = μ_q + σ_q · ε = 0.5 + 0.8 · 0.5 = 0.9
```

Paso 3 — Término de reconstrucción. El decoder predice $g(z) = 2 \cdot 0.9 = 1.8$:

```
log p(x|z) = −½ log(2π) − (x − g(z))²/2 = −0.9189 − (2.0 − 1.8)²/2
           = −0.9189 − 0.02 = −0.9389
```

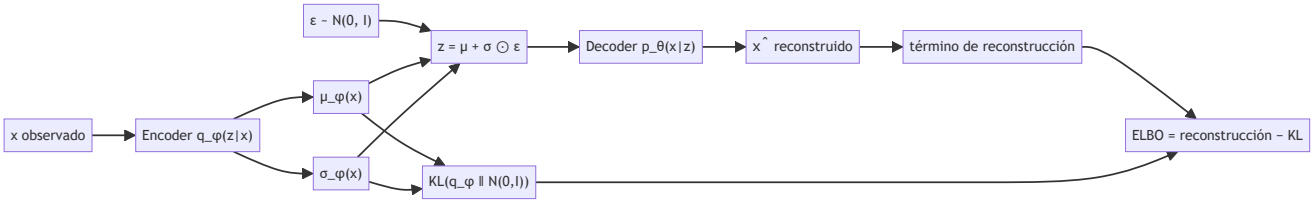
Paso 4 — ELBO (estimado con 1 muestra):

```
ELBO ≈ −0.9389 − 0.1681 = −1.1071   →   log p(x) ≥ −1.1071 (en esperanza)
```

Si el entrenamiento reduce el error de reconstrucción ($g(z)$ más cerca de x) sin inflar la KL, el ELBO sube y la cota sobre $\log p(x)$ se aprieta. Repite el cálculo con $\varepsilon = -0.5$: $z = 0.1$, $g(z) = 0.2$, $\log p(x|z) = -2.5389$ — la varianza entre muestras de ε es exactamente lo que promedia la esperanza E_q.

📊 Propiedades y comparación

Propiedad	VAE	Autoencoder clásico	GAN (clase 089)	Difusión (clase 090)
Objetivo	ELBO (cota de log-verosimilitud)	error de reconstrucción	juego minimax	cota variacional / ε-matching
Verosimilitud	cota inferior computable	no define densidad	implícita, no evaluable	cota computable
Muestreo	1 pasada del decoder	no genera (sin prior)	1 pasada del generador	decenas-miles de pasos
Espacio latente	continuo, regularizado	continuo, sin estructura garantizada	continuo (z del generador)	trayectoria de ruido
Fallo típico	muestras borrosas, posterior collapse	huecos latentes sin significado	colapso de modos	costo de muestreo
Entrenamiento	estable (un solo objetivo)	estable	inestable (dos redes)	estable



⚠️ Errores conceptuales frecuentes

1. **"El ELBO es la verosimilitud."** Es una cota inferior; la brecha es KL(q_φ(z|x) ‖ p_θ(z|x)). Comparar modelos por ELBO mezcla calidad del modelo y calidad de la aproximación variacional.
2. **"El encoder comprime como un ZIP."** q_φ(z|x) es una distribución, no un código determinista: el mismo x produce z distintos en cada muestreo, y la KL fuerza a que esa nube se parezca al prior.
3. **"La reparametrización es un detalle de implementación."** Es la contribución central: sin ella no hay gradiente de baja varianza a través del muestreo y el entrenamiento conjunto encoder-decoder no funciona en la práctica.
4. **"KL pequeña es siempre buena."** KL ≈ 0 en todas las dimensiones suele indicar posterior collapse: el decoder ignora z y el 'espacio latente' no representa nada.
5. **"Las muestras borrosas son un bug."** Son consecuencia estructural del decoder gaussiano/Bernoulli factorizado y del promedio que induce la verosimilitud; por eso VQ-VAE, β-VAE y los VAE jerárquicos modifican precisamente esas piezas.

🚀 Del aprendizaje a la operación

Entre este núcleo (un ELBO calculado a mano en 1D) y un VAE útil median: encoders y decoders convolucionales o transformer con millones de parámetros, ponderación del término KL (β-VAE, KL annealing) para controlar el colapso posterior, métricas de evaluación honestas (bits/dim, FID) además del ELBO, y en producción los VAE actúan sobre todo como compresores latentes dentro de sistemas mayores — el "latent" de Stable Diffusion es un autoencoder entrenado con este mismo marco.

🧪 Laboratorio

```
python lab.py
```

El laboratorio llama a `ai_evolution.labs.run_lab("generation")`. Esta decisión evita una implementación divergente por clase: cada una conserva un entrypoint propio, pero los motores didácticos se prueban como una biblioteca común.

🔍 Evidencia esperada

- tipo de laboratorio y semilla;
- entradas o decisiones observables;
- resultado estructurado;
- lista `evidence` con hechos que pueden inspeccionarse;
- lista `limitations` que impide presentar la demo como producción.

📓 Notebooks

- 📓 `notebook.ipynb`: recorrido guiado con la materia resumida.
- ✍️ `notebook_student.ipynb`: ejercicios para resolver.
- ✅ `notebook_solution.ipynb`: solución de referencia explicada.

📝 Evaluación

Criterio	Peso
Comprensión conceptual	25 %
Ejecución reproducible	25 %
Interpretación basada en evidencia	25 %
Riesgos, límites y mejora propuesta	25 %

Consulta assessment.md para preguntas y criterio de aceptación.

⚠️ Errores comunes

Síntoma	Causa probable	Corrección
El código corre, pero no hay conclusión	Se confundió ejecución con aprendizaje	Explica qué demuestra y qué no demuestra
El resultado cambia sin explicación	No se registró semilla o configuración	Conserva semilla, versión y parámetros
Se promete uso real	Se extrapoló desde una demo educativa	Declara entorno, datos, límites y revisión humana
Se copia una métrica aislada	No existe baseline ni costo de error	Añade comparación y criterio de decisión

❓ Preguntas frecuentes

¿Debo usar una API comercial?
No. El núcleo funciona localmente. Las extensiones LIVE se documentan por separado.

¿El laboratorio representa una implementación industrial?
No por sí solo. Enseña el contrato y el patrón; producción exige integración, seguridad, observabilidad, pruebas y operación.

¿Dónde profundizo?
Revisa las especializaciones enlazadas en el README raíz y la ruta siguiente.

🔗 Referencias

- Kingma, D. P. y Welling, M. (2013). *Auto-Encoding Variational Bayes*. Paper seminal del VAE. arXiv:1312.6114 — uso: fuente primaria del mecanismo estudiado
- Kingma, D. P. y Welling, M. (2019). *An Introduction to Variational Autoencoders*. Foundations and Trends in Machine Learning. arXiv:1906.02691 — uso: fuente primaria del mecanismo estudiado
- Doersch, C. (2016). *Tutorial on Variational Autoencoders*. arXiv:1606.05908 — uso: fuente primaria del mecanismo estudiado
- Goodfellow, I., Bengio, Y. y Courville, A. (2016). *Deep Learning*, cap. 20 (Deep Generative Models). deeplearningbook.org/contents/generative_models.html — uso: desarrollo extendido del tema
- Rezende, D. J., Mohamed, S. y Wierstra, D. (2014). *Stochastic Backpropagation and Approximate Inference in Deep Generative Models*. arXiv:1401.4082 — uso: fuente primaria del mecanismo estudiado
- Documentación de PyTorch: `torch.distributions.kl.kl_divergence` — uso: referencia consultada en su fuente original

---

📜 Papers que fundamentan esta clase

> Bloque generado por `python scripts/link_papers_to_classes.py`. La fuente es `papers/catalog/papers.json`.

Paper	Año	Qué desbloqueó	Miniatura
P38 · Bayes variacional con autocodificación	2013	Hace entrenable un modelo generativo latente: el truco de reparametrización deja pasar el gradiente a través del muestreo.	notebook

Cada ficha explica el problema anterior, la matemática mínima, los límites y los errores de atribución más frecuentes. Para leerlas con método: cómo leer un paper de IA · anexos matemáticos.

📚 Bibliografía de apoyo

> Bloque generado por `python scripts/link_sources_to_classes.py`. Cada obra lleva su localizador verificado en `sources/bibliography.json`.

Los papers dicen **de dónde salió** el mecanismo. Estas obras lo **desarrollan** con el espacio que una clase no tiene: teoría completa, demostraciones y ejercicios.

Obra	Edición	Localizador	Papel en esta clase
Goodfellow, Ian, Bengio, Yoshua y Courville, Aaron — *Deep Learning*	2016	ISBN 9780262035613 · web de la obra	citada en las referencias de esta clase · cap. 20 · obra de referencia de la parte 07

📝 Evaluación completa

❓ Preguntas

1. Define espacios latentes y autoencoders variacionales sin usar una marca o framework como definición.
2. Explica la relación entre VAE, latente, reconstrucción, muestreo.
3. Ejecuta `lab.py` dos veces con la misma semilla. ¿Qué debe conservarse?
4. Identifica una afirmación permitida y una afirmación exagerada sobre el resultado.
5. Propón una prueba negativa o un caso límite.

🏆 Reto verificable

Amplía el resultado del laboratorio con una clave `student_extension` que incluya:

- el supuesto que estás probando;
- una medición o comprobación;
- la conclusión;
- una limitación.

✅ Criterio de aceptación

- [ ] `lab.py` termina con código 0.
- [ ] El resultado contiene `kind`, `seed`, `evidence` y `limitations`.
- [ ] La extensión no modifica el comportamiento de otras clases.
- [ ] La interpretación referencia datos impresos por el laboratorio.
- [ ] Se declara al menos un riesgo o condición de no uso.

---

089 — GAN y entrenamiento adversarial

Parte: 07 — IA generativa para texto, imagen, audio, video y 3D
Nivel: avanzado · **Horas estimadas:** 6
Laboratorio: `generation` · **Estado:** `EXECUTABLE_CORE`

🎯 Propósito

Comprender **gan y entrenamiento adversarial** dentro de la evolución de la inteligencia artificial, implementar un experimento mínimo verificable y distinguir qué parte constituye evidencia frente a una afirmación todavía no comprobada.

📚 Resultados de aprendizaje

Al finalizar podrás:

1. Explicar gan y entrenamiento adversarial usando los conceptos `GAN`, `discriminador`, `generador`, `estabilidad`.
2. Ejecutar el laboratorio con una semilla explícita y revisar su contrato JSON.
3. Identificar al menos un supuesto, una limitación y un riesgo de aplicación.
4. Comparar el enfoque con la etapa anterior de la ruta de aprendizaje.
5. Producir una evidencia reproducible y una conclusión que no exceda los datos.

🧩 Conceptos centrales

`GAN`, `discriminador`, `generador`, `estabilidad`

🗺️ Ubicación en el mapa de la IA

Las redes generativas adversariales (Goodfellow et al., 2014) cambiaron la pregunta: en vez de maximizar una verosimilitud (como el VAE de la clase 088), dos redes compiten en un juego de suma cero y la calidad emerge del equilibrio. Entre 2014 y 2020 los GAN dominaron la síntesis de imágenes fotorrealistas (DCGAN, StyleGAN) y dejaron dos legados permanentes: el entrenamiento adversarial como técnica general (hoy reaparece en pérdidas perceptuales y en robustez) y la evidencia de que un modelo puede generar sin definir una densidad explícita. Los modelos de difusión (clase 090) los desplazaron precisamente atacando su punto débil: la inestabilidad.

📖 Fundamentos

⚔️ El juego minimax

Un GAN tiene dos redes con objetivos opuestos:

- **Generador** G: recibe ruido z ~ p(z) y produce muestras G(z) que intentan parecer datos reales.
- **Discriminador** D: recibe una muestra y estima la probabilidad D(x) ∈ (0, 1) de que sea real.

El objetivo original es un juego minimax sobre la función de valor V:

```
min_G max_D  V(D, G) = E_{x~p_data}[ log D(x) ] + E_{z~p(z)}[ log(1 - D(G(z))) ]
```

D maximiza V (acertar reales y falsos); G la minimiza (engañar a D). El entrenamiento alterna pasos de gradiente: k pasos de ascenso para D, un paso de descenso para G, sobre minibatches.

🎯 Discriminador óptimo y divergencia de Jensen-Shannon

Para G fijo, el discriminador óptimo tiene forma cerrada punto a punto:

```
D*(x) = p_data(x) / ( p_data(x) + p_g(x) )
```

donde p_g es la densidad de las muestras del generador. Sustituyendo D* en V se obtiene:

```
max_D V(D, G) = −log 4 + 2 · JSD( p_data ‖ p_g )
```

Es decir: con discriminador óptimo, entrenar G minimiza la **divergencia de Jensen-Shannon** entre la distribución real y la generada. El mínimo global se alcanza cuando p_g = p_data, y ahí D*(x) = 1/2 en todas partes (el discriminador queda reducido a lanzar una moneda) y V = −log 4 ≈ −1.386.

🔥 Non-saturating loss

En la práctica, al inicio del entrenamiento D distingue las muestras falsas con facilidad: D(G(z)) ≈ 0, y el gradiente de log(1 − D(G(z))) se satura (es casi plano cerca de 0). Goodfellow et al. proponen que G maximice log D(G(z)) en lugar de minimizar log(1 − D(G(z))):

```
Pérdida saturante de G:      L_G = E_z[ log(1 - D(G(z))) ]   (gradiente débil si D gana)
Pérdida non-saturating:      L_G = -E_z[ log D(G(z)) ]       (gradiente fuerte si D gana)
```

Ambas tienen el mismo punto fijo, pero la segunda da gradientes grandes justamente cuando el generador es malo — que es cuando más los necesita.

💥 Colapso de modos e inestabilidad

El equilibrio del juego es un punto de silla, no un mínimo: el descenso de gradiente simultáneo puede orbitar u oscilar sin converger. Fallos característicos:

- **Colapso de modos**: G concentra toda su masa en unas pocas muestras que engañan a D e ignora el resto de la distribución (genera "siempre el mismo dígito").
- **Discriminador demasiado fuerte**: si D llega a ser casi perfecto, sus gradientes hacia G se anulan y el aprendizaje se detiene.
- **Soportes disjuntos**: si p_data y p_g viven en variedades de baja dimensión que no se solapan, la JSD vale log 2 constante y no da señal útil — motivación del WGAN, que sustituye la JSD por la distancia de Wasserstein con un crítico 1-Lipschitz.

🧮 Ejemplo trabajado

Un paso del juego con números concretos. Minibatch: 2 muestras reales $x^{(1)}$, $x^{(2)}$ y 2 falsas $G(z^{(1)})$, $G(z^{(2)})$. Salidas actuales del discriminador:

```
D(x⁽¹⁾) = 0.9    D(x⁽²⁾) = 0.7        (reales: quiere valores altos)
D(G(z⁽¹⁾)) = 0.4    D(G(z⁽²⁾)) = 0.2  (falsas: quiere valores bajos)
```

Pérdida del discriminador (negativo de su objetivo, promediando cada mitad):

```
L_D = -½[log 0.9 + log 0.7] - ½[log(1-0.4) + log(1-0.2)]
    = -½[-0.1054 - 0.3567] - ½[-0.5108 - 0.2231]
    = 0.2310 + 0.3670 = 0.5980
```

Pérdida del generador, en sus dos variantes sobre las mismas falsas:

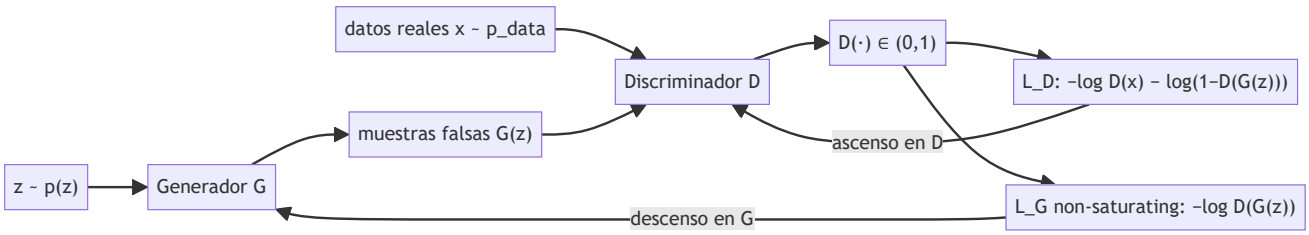
```
Non-saturating:  L_G = -½[log 0.4 + log 0.2] = -½[-0.9163 - 1.6094] = 1.2629
Saturante:       L_G = ½[log 0.6 + log 0.8] = -0.3670
```

La versión non-saturating asigna a la muestra peor puntuada (D = 0.2) un término de 1.6094 frente a 0.9163 de la otra: el gradiente empuja más fuerte donde G más falla. En el equilibrio ideal $D(\cdot) = 0.5$ en todo punto y $L_D = -2 \cdot \log 0.5$... es decir $L_D = 0.6931$ por muestra ($= \log 2$) y $V = -\log 4$.

Discriminador óptimo puntual: si en una región del espacio $p_{data} = 0.3$ y $p_g = 0.1$, entonces $D(x) = 0.3/(0.3 + 0.1) = 0.75$* — el valor 0.75 delata que ahí el generador pone menos masa de la que debería.

📊 Propiedades y comparación

Propiedad	GAN (original)	WGAN	VAE (clase 088)	Difusión (clase 090)
Objetivo	minimax ↔ JSD	distancia de Wasserstein	ELBO	cota variacional / ε-matching
Densidad p(x)	implícita, no evaluable	implícita	cota inferior computable	cota computable
Muestreo	1 pasada de G (rápido)	1 pasada	1 pasada del decoder	decenas-miles de pasos
Estabilidad	baja (punto de silla)	mejor (crítico Lipschitz)	alta	alta
Fallo típico	colapso de modos	recorte/penalización delicados	muestras borrosas	costo de muestreo
Señal con soportes disjuntos	nula (JSD constante)	sí (distancia geométrica)	n/a	n/a



⚠️ Errores conceptuales frecuentes

1. **"El GAN aprende una densidad y luego muestrea."** No define p_g de forma evaluable: solo sabe transformar ruido en muestras. No se puede calcular la verosimilitud de un dato bajo un GAN estándar.
2. **"Pérdidas bajando = entrenamiento sano."** En un juego, las pérdidas de D y G son relativas al oponente: pueden oscilar en un GAN que funciona y estancarse en uno colapsado. Se valida mirando muestras y métricas (FID), no solo curvas.
3. **"Conviene entrenar D hasta el óptimo en cada paso."** Con D casi perfecto el gradiente hacia G se desvanece (con la pérdida saturante) o la señal JSD es constante con soportes disjuntos. El balance D/G es parte del diseño.
4. **"El colapso de modos es underfitting."** G puede producir muestras nítidas y de alta calidad y aun así cubrir una fracción mínima de la distribución: es un fallo de cobertura, no de capacidad.
5. **"D*(x) = 1/2 significa que el discriminador falló."** Al contrario: es la firma del equilibrio p_g = p_data; si D no puede hacer nada mejor que el azar, el generador ganó el juego.

🚀 Del aprendizaje a la operación

Entre este juego calculado a mano y un StyleGAN operativo median: arquitecturas convolucionales con normalización espectral y regularización de gradiente (R1), trucos de balance D/G y schedules ajustados empíricamente, métricas de evaluación (FID, precision/recall de distribuciones) con sus propios sesgos, y datasets curados a gran escala. Además, la capacidad de sintetizar caras realistas convierte la procedencia del contenido (clase 098) en requisito operativo, no en opción.

🧪 Laboratorio

```
python lab.py
```

El laboratorio llama a `ai_evolution.labs.run_lab("generation")`. Esta decisión evita una implementación divergente por clase: cada una conserva un entrypoint propio, pero los motores didácticos se prueban como una biblioteca común.

🔍 Evidencia esperada

- tipo de laboratorio y semilla;
- entradas o decisiones observables;
- resultado estructurado;
- lista `evidence` con hechos que pueden inspeccionarse;

- lista `limitations` que impide presentar la demo como producción.

📓 Notebooks

- 📓 `notebook.ipynb`: recorrido guiado con la materia resumida.
- ✍️ `notebook_student.ipynb`: ejercicios para resolver.
- ✅ `notebook_solution.ipynb`: solución de referencia explicada.

📝 Evaluación

Criterio	Peso
Comprensión conceptual	25 %
Ejecución reproducible	25 %
Interpretación basada en evidencia	25 %
Riesgos, límites y mejora propuesta	25 %

Consulta assessment.md para preguntas y criterio de aceptación.

⚠️ Errores comunes

Síntoma	Causa probable	Corrección
El código corre, pero no hay conclusión	Se confundió ejecución con aprendizaje	Explica qué demuestra y qué no demuestra
El resultado cambia sin explicación	No se registró semilla o configuración	Conserva semilla, versión y parámetros
Se promete uso real	Se extrapoló desde una demo educativa	Declara entorno, datos, límites y revisión humana
Se copia una métrica aislada	No existe baseline ni costo de error	Añade comparación y criterio de decisión

❓ Preguntas frecuentes

¿Debo usar una API comercial?
No. El núcleo funciona localmente. Las extensiones LIVE se documentan por separado.

¿El laboratorio representa una implementación industrial?
No por sí solo. Enseña el contrato y el patrón; producción exige integración, seguridad, observabilidad, pruebas y operación.

¿Dónde profundizo?
Revisa las especializaciones enlazadas en el README raíz y la ruta siguiente.

🔗 Referencias

- Goodfellow, I. et al. (2014). *Generative Adversarial Networks*. Paper seminal. arXiv:1406.2661 — uso: fuente primaria del mecanismo estudiado
- Arjovsky, M., Chintala, S. y Bottou, L. (2017). *Wasserstein GAN*. arXiv:1701.07875 — uso: fuente primaria del mecanismo estudiado
- Radford, A., Metz, L. y Chintala, S. (2015). *Unsupervised Representation Learning with Deep Convolutional GANs* (DCGAN). arXiv:1511.06434 — uso: fuente primaria del mecanismo estudiado
- Goodfellow, I. (2016). *NIPS 2016 Tutorial: Generative Adversarial Networks*. arXiv:1701.00160 — uso: fuente primaria del mecanismo estudiado
- Goodfellow, I., Bengio, Y. y Courville, A. (2016). *Deep Learning*, cap. 20 (Deep Generative Models). deeplearningbook.org/contents/generative_models.html — uso: desarrollo extendido del tema
- Documentación de PyTorch: tutorial oficial DCGAN — uso: referencia consultada en su fuente original

---

📜 Papers que fundamentan esta clase

> Bloque generado por `python scripts/link_papers_to_classes.py`. La fuente es `papers/catalog/papers.json`.

Paper	Año	Qué desbloqueó	Miniatura
P39 · Redes generativas adversarias	2014	Convierte la generación en un juego: dos redes compiten y ninguna necesita una verosimilitud explícita.	notebook

Cada ficha explica el problema anterior, la matemática mínima, los límites y los errores de atribución más frecuentes. Para leerlas con método: cómo leer un paper de IA · anexos matemáticos.

📚 Bibliografía de apoyo

> Bloque generado por `python scripts/link_sources_to_classes.py`. Cada obra lleva su localizador verificado en `sources/bibliography.json`.

Los papers dicen **de dónde salió** el mecanismo. Estas obras lo **desarrollan** con el espacio que una clase no tiene: teoría completa, demostraciones y ejercicios.

Obra	Edición	Localizador	Papel en esta clase
Goodfellow, Ian, Bengio, Yoshua y Courville, Aaron — *Deep Learning*	2016	ISBN 9780262035613 · web de la obra	citada en las referencias de esta clase · cap. 20 · obra de referencia de la parte 07

📝 Evaluación completa

❓ Preguntas

1. Define gan y entrenamiento adversarial sin usar una marca o framework como definición.
2. Explica la relación entre GAN, discriminador, generador, estabilidad.
3. Ejecuta `lab.py` dos veces con la misma semilla. ¿Qué debe conservarse?
4. Identifica una afirmación permitida y una afirmación exagerada sobre el resultado.
5. Propón una prueba negativa o un caso límite.

🏆 Reto verificable

Amplía el resultado del laboratorio con una clave `student_extension` que incluya:

- el supuesto que estás probando;
- una medición o comprobación;
- la conclusión;
- una limitación.

✅ Criterio de aceptación

- [ ] `lab.py` termina con código 0.
- [ ] El resultado contiene `kind`, `seed`, `evidence` y `limitations`.
- [ ] La extensión no modifica el comportamiento de otras clases.
- [ ] La interpretación referencia datos impresos por el laboratorio.
- [ ] Se declara al menos un riesgo o condición de no uso.

---

090 — Modelos de difusión

Parte: 07 — IA generativa para texto, imagen, audio, video y 3D
Nivel: avanzado · **Horas estimadas:** 6
Laboratorio: `generation` · **Estado:** `EXECUTABLE_CORE`

🎯 Propósito

Comprender **modelos de difusión** dentro de la evolución de la inteligencia artificial, implementar un experimento mínimo verificable y distinguir qué parte constituye evidencia frente a una afirmación todavía no comprobada.

📚 Resultados de aprendizaje

Al finalizar podrás:

1. Explicar modelos de difusión usando los conceptos `DDPM`, `denoising`, `scheduler`, `sampling`.
2. Ejecutar el laboratorio con una semilla explícita y revisar su contrato JSON.
3. Identificar al menos un supuesto, una limitación y un riesgo de aplicación.
4. Comparar el enfoque con la etapa anterior de la ruta de aprendizaje.
5. Producir una evidencia reproducible y una conclusión que no exceda los datos.

🧩 Conceptos centrales

`DDPM`, `denoising`, `scheduler`, `sampling`

🗺️ Ubicación en el mapa de la IA

Los modelos de difusión nacen en la termodinámica fuera de equilibrio (Sohl-Dickstein et al., 2015) y maduran con DDPM (Ho et al., 2020), que demostró calidad de imagen comparable o superior a los GAN con un entrenamiento estable de un solo objetivo. Combinan lo mejor de las dos clases anteriores: como el VAE (088) optimizan una cota variacional, y como el GAN (089) producen muestras nítidas — sin juego adversarial. Son el motor de Stable Diffusion, DALL·E, Imagen y de los generadores de audio y video de las clases siguientes, donde el condicionamiento por texto (091) se monta sobre exactamente este proceso.

📖 Fundamentos

🌫️ Proceso forward: destruir con ruido gaussiano

El proceso forward es una cadena de Markov fija (sin parámetros aprendidos) que corrompe el dato x_0 en T pasos añadiendo ruido gaussiano según un **scheduler de varianzas** $\beta_1, \ldots, \beta_T$ (típicamente crecientes, p. ej. de 10^{-4} a 0.02 con $T = 1000$):

```
q(x_t | x_{t-1}) = N( x_t ; √(1-β_t) · x_{t-1} ,  β_t · I )
```

El factor √(1−β_t) encoge la señal exactamente lo necesario para que la varianza total se conserve. Definiendo α_t = 1 − β_t y ᾱ_t = ∏_{s=1}^{t} α_s, la composición de t pasos gaussianos tiene **forma cerrada** — se puede saltar de x0 a x_t sin recorrer la cadena:

```
q(x_t | x₀) = N( x_t ; √ᾱ_t · x₀ ,  (1−ᾱ_t) · I )
x_t = √ᾱ_t · x₀ + √(1−ᾱ_t) · ε ,   ε ~ N(0, I)
```

Como ᾱ_T → 0, al final x_T es indistinguible de ruido puro N(0, I): la distribución de partida del muestreo es conocida.

🔁 Proceso reverse: aprender a des-ruido

Generar es invertir la cadena: partir de x_T ~ N(0, I) y quitar ruido paso a paso. La inversa exacta q(x_{t−1}|x_t) es intratable (requiere la densidad de los datos), pero para β_t pequeños es aproximadamente gaussiana, así que se aprende:

```
p_θ(x_{t-1} | x_t) = N( x_{t-1} ; μ_θ(x_t, t) ,  σ_t² · I )
```

La observación clave de DDPM: en lugar de predecir μ directamente, la red predice **el ruido ε** que se usó para llegar a x_t. De la forma cerrada se despeja x0 = (x_t − √(1−ᾱ_t)·ε)/√ᾱ_t, y la media óptima queda:

```
μ_θ(x_t, t) = (1/√α_t) · ( x_t − β_t/√(1−ᾱ_t) · ε_θ(x_t, t) )
```

🎯 Objetivo simplificado

La derivación completa parte de la cota variacional (un ELBO como el del VAE, con un término KL por paso), pero Ho et al. muestran que descartar los pesos de cada término funciona mejor en la práctica. El objetivo de entrenamiento se reduce a:

```
L_simple = E_{x₀, t, ε} [ ‖ ε − ε_θ( √ᾱ_t·x₀ + √(1−ᾱ_t)·ε ,  t ) ‖² ]
```

El algoritmo de entrenamiento cabe en cuatro líneas:

```
repetir:
    x₀ ~ datos;  t ~ Uniforme{1..T};  ε ~ N(0, I)
    x_t = √ᾱ_t·x₀ + √(1−ᾱ_t)·ε
    paso de gradiente sobre ‖ε − ε_θ(x_t, t)‖²
```

Una sola red ε_θ (una U-Net o un transformer, condicionada en t) aprende todos los niveles de ruido. No hay juego adversarial ni riesgo de colapso de modos: es una regresión estable. Este objetivo equivale además a aprender la **función de score** ∇_x log q(x_t) (Song y Ermon), lo que conecta difusión con score matching.

⏱️ Muestreo y el costo de iterar

Generar exige recorrer la cadena reverse: T evaluaciones de la red (1000 en DDPM original) frente a 1 sola pasada en GAN o VAE. DDIM reformula el proceso como no markoviano y determinista, permitiendo saltar

pasos (50-100 evaluaciones con poca pérdida); la destilación posterior lo reduce a unas pocas. El scheduler de β y el número de pasos de muestreo son los dos diales operativos principales.

🧮 Ejemplo trabajado

Difusión escalar (1D) con dos pasos y scheduler $\beta_1 = 0.1$, $\beta_2 = 0.2$.

Paso 1 — alfas acumuladas:

```
α₁ = 1 − 0.1 = 0.9          α₂ = 1 − 0.2 = 0.8
ᾱ₁ = 0.9                    ᾱ₂ = 0.9 · 0.8 = 0.72
```

Paso 2 — forward directo a t = 2 con $x_0 = 1.0$ y ruido muestreado $\varepsilon = 0.5$:

```
x₂ = √ᾱ₂ · x₀ + √(1−ᾱ₂) · ε
   = √0.72 · 1.0 + √0.28 · 0.5
   = 0.8485 + 0.5292 · 0.5
   = 0.8485 + 0.2646 = 1.1131
```

La señal conserva el 84.85 % de su escala y el ruido aporta desviación $\sqrt{0.28} \approx 0.53$. Con T grande, $\sqrt{\bar{\alpha}_T} \to 0$ y x_T sería prácticamente ε puro.

Paso 3 — reconstrucción de x_0 si la red acierta el ruido. Un ε_θ perfecto devolvería $\hat{\varepsilon} = 0.5$ y podríamos despejar:

```
x̂₀ = ( x₂ − √(1−ᾱ₂) · ε̂ ) / √ᾱ₂ = ( 1.1131 − 0.2646 ) / 0.8485 = 1.0000 ✓
```

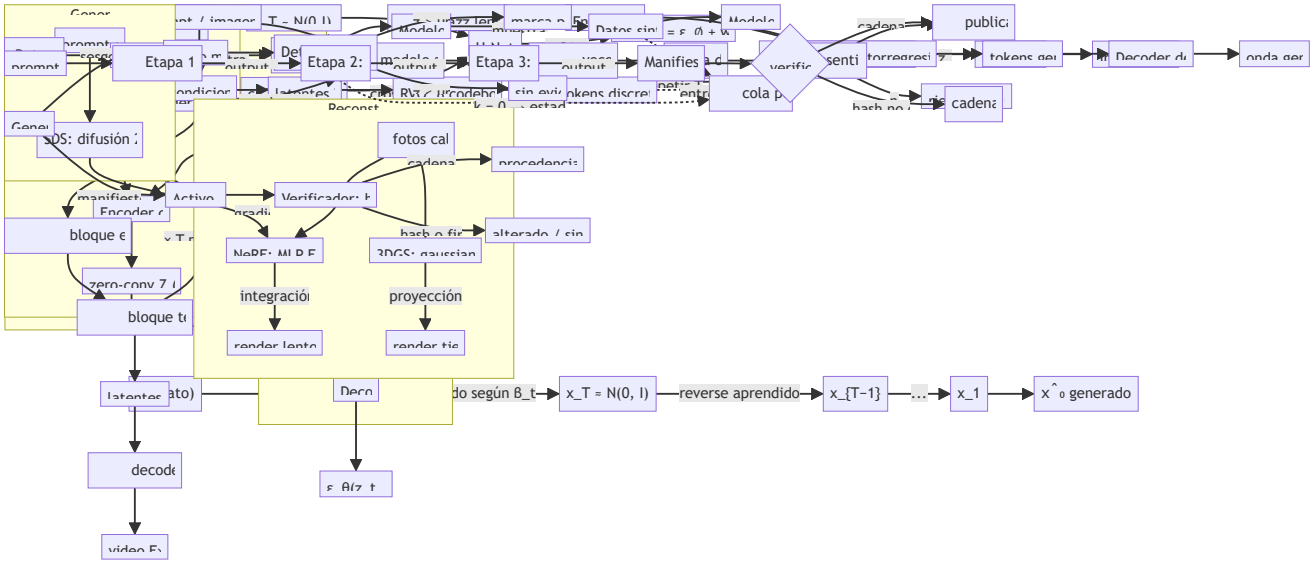
Paso 4 — la pérdida castiga el error de ruido. Si la red predice $\hat{\varepsilon} = 0.3$:

```
L_simple = (0.5 − 0.3)² = 0.04
x̂₀ = (1.1131 − 0.5292·0.3)/0.8485 = 1.1247   (error de 0.1247 en el dato)
```

El mismo error de ruido produce un error en $\hat{x}_0$ amplificado por $\sqrt{1-\bar{\alpha}_t}/\sqrt{\bar{\alpha}_t}$ — por eso los pasos muy ruidosos (t alto) son los más difíciles y los que fijan la estructura global de la muestra.

📊 Propiedades y comparación

Propiedad	DDPM	DDIM	GAN (089)	VAE (088)
Objetivo	‖ε − ε_θ‖² (cota variacional simplificada)	mismo entrenamiento	juego minimax	ELBO
Muestreo	~1000 pasos estocásticos	20-100 pasos, determinista	1 pasada	1 pasada
Estabilidad de entrenamiento	alta (regresión)	alta	baja	alta
Cobertura de modos	alta	alta	riesgo de colapso	alta
Verosimilitud	cota computable	cota computable	no evaluable	cota computable
Fallo típico	costo de muestreo	leve pérdida de diversidad	colapso de modos	borrosidad



⚠️ Errores conceptuales frecuentes

1. **"El modelo aprende a quitar todo el ruido de golpe."** ε_θ predice el ruido total presente en x_t, pero el muestreo solo retrocede un paso cada vez; la estimación de x0 se rehace y refina en cada iteración.
2. **"El proceso forward se entrena."** No tiene parámetros: es una cadena fija definida por el scheduler β. Solo la red reverse ε_θ aprende.
3. **"Más pasos de muestreo siempre mejoran la calidad."** Hay rendimientos decrecientes rápidos; DDIM logra con 50 pasos casi lo mismo que DDPM con 1000. El número de pasos es un trade-off calidad/latencia, no una virtud en sí.
4. **"La pérdida ‖ε − ε̂‖² es un truco sin justificación."** Es la cota variacional con los pesos por paso igualados a 1; la teoría (y su equivalencia con score matching) está completa en Ho et al. y Song et al.
5. **"Difusión reemplaza a VAE y GAN en todo."** Paga su calidad con decenas de evaluaciones de red por muestra; en latencia estricta un generador de una pasada sigue ganando, y los sistemas reales (Stable Diffusion) usan un VAE por debajo.

🚀 Del aprendizaje a la operación

Entre este forward escalar a mano y un generador real median: una U-Net o DiT con cientos de millones de parámetros condicionada en t (y en texto, clase 091), schedulers coseno y espacios latentes comprimidos para abaratar cada paso, muestreadores avanzados (DDIM, DPM-Solver) y destilación para bajar de 1000 a menos de 10 evaluaciones, y guías de clasificador libre para controlar la generación. La calidad fotorrealista resultante hace obligatoria la trazabilidad de procedencia que cierra esta parte (clase 098).

🧪 Laboratorio

```
python lab.py
```

El laboratorio llama a `ai_evolution.labs.run_lab("generation")`. Esta decisión evita una implementación divergente por clase: cada una conserva un entrypoint propio, pero los motores didácticos se prueban como una biblioteca común.

🔍 Evidencia esperada

- tipo de laboratorio y semilla;
- entradas o decisiones observables;
- resultado estructurado;
- lista `evidence` con hechos que pueden inspeccionarse;
- lista `limitations` que impide presentar la demo como producción.

📓 Notebooks

- 📓 `notebook.ipynb`: recorrido guiado con la materia resumida.
- ✍️ `notebook_student.ipynb`: ejercicios para resolver.
- ✅ `notebook_solution.ipynb`: solución de referencia explicada.

📝 Evaluación

Criterio	Peso
Comprensión conceptual	25 %
Ejecución reproducible	25 %
Interpretación basada en evidencia	25 %
Riesgos, límites y mejora propuesta	25 %

Consulta assessment.md para preguntas y criterio de aceptación.

⚠️ Errores comunes

Síntoma	Causa probable	Corrección
El código corre, pero no hay conclusión	Se confundió ejecución con aprendizaje	Explica qué demuestra y qué no demuestra
El resultado cambia sin explicación	No se registró semilla o configuración	Conserva semilla, versión y parámetros
Se promete uso real	Se extrapoló desde una demo educativa	Declara entorno, datos, límites y revisión humana
Se copia una métrica aislada	No existe baseline ni costo de error	Añade comparación y criterio de decisión

❓ Preguntas frecuentes

¿Debo usar una API comercial?
No. El núcleo funciona localmente. Las extensiones LIVE se documentan por separado.

¿El laboratorio representa una implementación industrial?
No por sí solo. Enseña el contrato y el patrón; producción exige integración, seguridad, observabilidad, pruebas y operación.

¿Dónde profundizo?
Revisa las especializaciones enlazadas en el README raíz y la ruta siguiente.

🔗 Referencias

- Ho, J., Jain, A. y Abbeel, P. (2020). *Denoising Diffusion Probabilistic Models*. Paper seminal de DDPM. arXiv:2006.11239 — uso: fuente primaria del mecanismo estudiado
- Sohl-Dickstein, J. et al. (2015). *Deep Unsupervised Learning using Nonequilibrium Thermodynamics*. Origen del marco de difusión. arXiv:1503.03585 — uso: fuente primaria del mecanismo estudiado
- Song, J., Meng, C. y Ermon, S. (2020). *Denoising Diffusion Implicit Models* (DDIM). arXiv:2010.02502 — uso: fuente primaria del mecanismo estudiado
- Song, Y. et al. (2020). *Score-Based Generative Modeling through Stochastic Differential Equations*. arXiv:2011.13456 — uso: fuente primaria del mecanismo estudiado
- Rombach, R. et al. (2021). *High-Resolution Image Synthesis with Latent Diffusion Models* (Stable Diffusion). arXiv:2112.10752 — uso: fuente primaria del mecanismo estudiado
- Documentación de Hugging Face Diffusers: `DDPMScheduler` — uso: referencia consultada en su fuente original

---

📜 Papers que fundamentan esta clase

> Bloque generado por `python scripts/link_papers_to_classes.py`. La fuente es `papers/catalog/papers.json`.

Paper	Año	Qué desbloqueó	Miniatura
P17 · Modelos probabilísticos de difusión con eliminación de ruido	2020	La generación deja de ser un salto en la oscuridad: se aprende a deshacer, paso a paso, un proceso de ruido conocido.	notebook

Cada ficha explica el problema anterior, la matemática mínima, los límites y los errores de atribución más frecuentes. Para leerlas con método: cómo leer un paper de IA · anexos matemáticos.

📚 Bibliografía de apoyo

> Bloque generado por `python scripts/link_sources_to_classes.py`. Cada obra lleva su localizador verificado en `sources/bibliography.json`.

Los papers dicen **de dónde salió** el mecanismo. Estas obras lo **desarrollan** con el espacio que una clase no tiene: teoría completa, demostraciones y ejercicios.

Obra	Edición	Localizador	Papel en esta clase
Goodfellow, Ian, Bengio, Yoshua y Courville, Aaron — *Deep Learning*	2016	ISBN 9780262035613 · web de la obra	obra de referencia de la parte 07 · capítulos de modelos generativos

📝 Evaluación completa

❓ Preguntas

1. Define modelos de difusión sin usar una marca o framework como definición.
2. Explica la relación entre DDPM, denoising, scheduler, sampling.
3. Ejecuta `lab.py` dos veces con la misma semilla. ¿Qué debe conservarse?
4. Identifica una afirmación permitida y una afirmación exagerada sobre el resultado.
5. Propón una prueba negativa o un caso límite.

🏆 Reto verificable

Amplía el resultado del laboratorio con una clave `student_extension` que incluya:

- el supuesto que estás probando;
- una medición o comprobación;
- la conclusión;
- una limitación.

✅ Criterio de aceptación

- [ ] `lab.py` termina con código 0.

- [ ] El resultado contiene `kind`, `seed`, `evidence` y `limitations`.
- [ ] La extensión no modifica el comportamiento de otras clases.
- [ ] La interpretación referencia datos impresos por el laboratorio.
- [ ] Se declara al menos un riesgo o condición de no uso.

---

091 — Texto a imagen y condicionamiento

Parte: 07 — IA generativa para texto, imagen, audio, video y 3D
Nivel: avanzado · **Horas estimadas:** 6
Laboratorio: `generation` · **Estado:** `EXECUTABLE_CORE`

🎯 Propósito

Comprender **texto a imagen y condicionamiento** dentro de la evolución de la inteligencia artificial, implementar un experimento mínimo verificable y distinguir qué parte constituye evidencia frente a una afirmación todavía no comprobada.

📚 Resultados de aprendizaje

Al finalizar podrás:

1. Explicar texto a imagen y condicionamiento usando los conceptos `text-to-image`, `conditioning`, `guidance`, `prompt`.
2. Ejecutar el laboratorio con una semilla explícita y revisar su contrato JSON.
3. Identificar al menos un supuesto, una limitación y un riesgo de aplicación.
4. Comparar el enfoque con la etapa anterior de la ruta de aprendizaje.
5. Producir una evidencia reproducible y una conclusión que no exceda los datos.

🧩 Conceptos centrales

`text-to-image`, `conditioning`, `guidance`, `prompt`

🗺️ Ubicación en el mapa de la IA

Los modelos de difusión de la clase 090 generan imágenes, pero sin control: muestrean de la distribución completa aprendida. Esta clase añade la pieza que convirtió la difusión en un producto masivo (2021-2022): **condicionar** la generación con texto mediante cross-attention y **guiarla** con classifier-free guidance, todo dentro del espacio latente de un autoencoder para que sea computacionalmente viable (Stable Diffusion). Estas mismas técnicas de condicionamiento habilitan la clase 092 (control estructural con ControlNet) y se reutilizan en generación de video y 3D.

📖 Fundamentos

📝 Del prompt al vector de condición

El texto no entra "crudo" al modelo. Un encoder de texto preentrenado —en Stable Diffusion, el text encoder de **CLIP** (Radford et al., 2021)— tokeniza el prompt y produce una secuencia de embeddings $c = (c_1, \ldots, c_L)$, un vector por token (en SD 1.x, $L = 77$ tokens de 768 dimensiones). CLIP se entrenó con contraste imagen-

texto sobre cientos de millones de pares, así que sus embeddings ya codifican semántica visual: "un gato naranja" queda cerca de fotos de gatos naranjas en el espacio conjunto.

🔀 Condicionamiento por cross-attention

La U-Net de difusión predice el ruido ε_θ(x_t, t, c). El texto se inyecta en capas de **cross-attention** intercaladas en la U-Net: las queries Q vienen de las features espaciales de la imagen ruidosa, y las keys K y values V vienen de los embeddings del texto:

```
Attention(Q, K, V) = softmax(Q·Kᵀ / √d) · V
Q = W_Q · ϕ(x_t)      (features de la imagen, una query por posición espacial)
K = W_K · c,  V = W_V · c   (una key/value por token del prompt)
```

Cada posición de la imagen "consulta" qué tokens del prompt le son relevantes: los pesos de atención alinean regiones espaciales con palabras. Por eso los mapas de atención permiten localizar qué píxeles atendieron a "gato" y cuáles a "naranja".

🎯 Classifier-free guidance (CFG)

Condicionar no basta: el modelo tiende a ignorar parcialmente el prompt. Ho y Salimans (2022) propusieron entrenar el mismo modelo con y sin condición (durante el entrenamiento se reemplaza c por el token vacío ∅ con probabilidad ~10-20 %) y, en inferencia, **extrapolar** la predicción en la dirección que marca el texto:

```
ε̃ = ε_θ(x_t, ∅) + w · (ε_θ(x_t, c) − ε_θ(x_t, ∅))
```

- w = 1 recupera la predicción condicional pura.
- w > 1 (típico: 7-8) amplifica la dirección "lo que el texto añade sobre lo incondicional": más adherencia al prompt, menos diversidad, y con w muy alto colores sobresaturados y artefactos.
- Cuesta **dos pasadas** de la U-Net por paso de muestreo (una con c, otra con ∅).

🗜️ Latent diffusion: difundir en el espacio de un autoencoder

Difundir en píxeles 512×512 es carísimo: cada paso de la U-Net opera sobre 786 432 valores. Rombach et al. (2022) entrenan primero un **autoencoder** (VAE con pérdida perceptual y adversarial) que comprime la imagen x a un latente z = E(x) de 64×64×4, y ejecutan **toda la difusión en z**; al final, el decoder D(z₀) reconstruye la imagen. El factor de reducción espacial es f = 8 por lado. La apuesta empírica: la difusión se encarga de la composición semántica y el autoencoder de los detalles perceptuales de alta frecuencia — separar ambos ahorra un orden de magnitud de cómputo sin perder calidad apreciable.

```
Pipeline de Stable Diffusion (inferencia):
1. c = CLIP_text(prompt);  z_T ~ N(0, I)   (latente 64×64×4)
2. para t = T … 1:
       ε̃ = ε_θ(z_t, ∅) + w·(ε_θ(z_t, c) − ε_θ(z_t, ∅))    # CFG, 2 pasadas
       z_{t-1} = paso_de_muestreo(z_t, ε̃, t)               # DDPM/DDIM
3. imagen = D(z₀)                                          # decoder del VAE
```

🧮 Ejemplo trabajado

CFG con números concretos. Supón que en cierto paso, para una coordenada del latente, el modelo predice ruido incondicional $\varepsilon_\varnothing = 0.20$ y condicional $\varepsilon_c = 0.32$ (el prompt "empuja" esa coordenada). Con $w = 7.5$:

```
ε̃ = 0.20 + 7.5 · (0.32 − 0.20) = 0.20 + 7.5 · 0.12 = 0.20 + 0.90 = 1.10
```

La diferencia condicional era +0.12; la guía la amplifica a +0.90. Nota que $\tilde{\varepsilon} = 1.10$ queda **fuera** del intervalo [0.20, 0.32]: CFG extrapola, no interpola — por eso w grande produce imágenes "quemadas". Con $w = 1$: $\tilde{\varepsilon} = 0.32$ (condicional pura).

Ahorro de latent diffusion. Dimensión del problema por paso de difusión:

```
Píxeles:  512 × 512 × 3 = 786 432 valores
Latente:   64 ×  64 × 4 =  16 384 valores    → factor 48× menos entradas
```

Como el costo de las convoluciones escala con el área espacial, pasar de 512^2 a 64^2 reduce ~64× las operaciones espaciales por capa. Con 50 pasos de muestreo y 2 pasadas por CFG son 100 evaluaciones de U-Net: hacerlas sobre 16 384 valores en vez de 786 432 es la diferencia entre segundos y minutos por imagen en una misma GPU. El precio: una sola pasada extra de encoder/decoder del VAE y los artefactos propios de su reconstrucción (texto pequeño, tramas finas).

📊 Propiedades y comparación

Enfoque	Espacio	Condicionamiento	Costo de muestreo	Trade-off principal
Difusión en píxeles (DDPM, Imagen)	píxeles	cross-attention (+ cascada de superresolución)	muy alto (T pasos sobre imagen completa)	máxima fidelidad, cómputo prohibitivo
Latent diffusion (Stable Diffusion)	latente VAE 64×64×4	cross-attention + CFG	alto pero ~48× menor por paso	artefactos del decoder del VAE
Autoregresivo sobre tokens (DALL·E 1, Parti)	tokens discretos VQ	prefijo de texto en el transformer	O(n) tokens secuenciales	orden de generación artificial en 2D
GAN condicional (clase 089)	píxeles	vector/embedding en G y D	1 pasada (rapidísimo)	entrenamiento inestable, menor diversidad
DALL·E 2 (prior + decoder)	embedding CLIP → difusión	prior texto→imagen-embedding	alto	dos etapas: errores del prior se propagan

⚠️ Errores conceptuales frecuentes

1. **"El modelo entiende el prompt como un LLM."** No: el encoder CLIP produce embeddings contrastivos con límite de 77 tokens; composiciones complejas ("el cubo rojo *sobre* la esfera azul") fallan a menudo porque la atención no garantiza vinculación correcta atributo-objeto.
2. **"Subir el guidance siempre mejora la imagen."** w alto aumenta adherencia al prompt pero reduce diversidad y satura colores; es una extrapolación, no un ajuste fino. Valores útiles suelen estar en 5-10.
3. **"La difusión ocurre sobre la imagen."** En latent diffusion ocurre sobre z (64×64×4); la imagen solo existe tras el decoder. Por eso defectos como texto ilegible son en parte atribuibles al VAE, no a la difusión.
4. **"CFG usa un clasificador."** Es *classifier-free* precisamente porque sustituye al clasificador externo del classifier guidance original (Dhariwal y Nichol) por la diferencia entre predicción condicional e incondicional del propio modelo.
5. **"La semilla fija garantiza la misma imagen en cualquier entorno."** Fija el ruido inicial, pero cambios de versión del modelo, scheduler, precisión numérica o hardware alteran el resultado: la reproducibilidad exige fijar todo el stack.

🚀 Del aprendizaje a la operación

Entre este núcleo y un servicio real de texto-a-imagen faltan: filtros de seguridad de entrada (prompt) y de salida (clasificador de contenido), gestión de derechos sobre los datos de entrenamiento y las salidas, procedencia verificable de lo generado (C2PA/watermarking), optimización de inferencia (destilación de pasos, cuantización) para servir a escala, y evaluación humana sistemática — FID y CLIP score no capturan fallos de composición ni sesgos de representación.

🧪 Laboratorio

```
python lab.py
```

El laboratorio llama a `ai_evolution.labs.run_lab("generation")`. Esta decisión evita una implementación divergente por clase: cada una conserva un entrypoint propio, pero los motores didácticos se prueban como una biblioteca común.

🔍 Evidencia esperada

- tipo de laboratorio y semilla;
- entradas o decisiones observables;
- resultado estructurado;
- lista `evidence` con hechos que pueden inspeccionarse;
- lista `limitations` que impide presentar la demo como producción.

📓 Notebooks

- 📓 `notebook.ipynb`: recorrido guiado con la materia resumida.
- ✍️ `notebook_student.ipynb`: ejercicios para resolver.
- ✅ `notebook_solution.ipynb`: solución de referencia explicada.

📝 Evaluación

Criterio	Peso
Comprensión conceptual	25 %
Ejecución reproducible	25 %
Interpretación basada en evidencia	25 %
Riesgos, límites y mejora propuesta	25 %

Consulta assessment.md para preguntas y criterio de aceptación.

⚠️ Errores comunes

Síntoma	Causa probable	Corrección
El código corre, pero no hay conclusión	Se confundió ejecución con aprendizaje	Explica qué demuestra y qué no demuestra
El resultado cambia sin explicación	No se registró semilla o configuración	Conserva semilla, versión y parámetros
Se promete uso real	Se extrapoló desde una demo educativa	Declara entorno, datos, límites y revisión humana
Se copia una métrica aislada	No existe baseline ni costo de error	Añade comparación y criterio de decisión

❓ Preguntas frecuentes

¿Debo usar una API comercial?
No. El núcleo funciona localmente. Las extensiones LIVE se documentan por separado.

¿El laboratorio representa una implementación industrial?
No por sí solo. Enseña el contrato y el patrón; producción exige integración, seguridad, observabilidad, pruebas y operación.

¿Dónde profundizo?
Revisa las especializaciones enlazadas en el README raíz y la ruta siguiente.

🔗 Referencias

- Rombach, R., Blattmann, A., Lorenz, D., Esser, P. y Ommer, B. (2022). *High-Resolution Image Synthesis with Latent Diffusion Models* (Stable Diffusion). arXiv:2112.10752 — uso: fuente primaria del mecanismo estudiado
- Ho, J. y Salimans, T. (2022). *Classifier-Free Diffusion Guidance*. arXiv:2207.12598 — uso: fuente primaria del mecanismo estudiado
- Radford, A. et al. (2021). *Learning Transferable Visual Models From Natural Language Supervision* (CLIP). arXiv:2103.00020 — uso: fuente primaria del mecanismo estudiado
- Ramesh, A. et al. (2022). *Hierarchical Text-Conditional Image Generation with CLIP Latents* (DALL·E 2). arXiv:2204.06125 — uso: fuente primaria del mecanismo estudiado
- Documentación de Hugging Face Diffusers: Stable Diffusion pipeline — uso: referencia consultada en su fuente original

---

📜 Papers que fundamentan esta clase

> Bloque generado por `python scripts/link_papers_to_classes.py`. La fuente es `papers/catalog/papers.json`.

Paper	Año	Qué desbloqueó	Miniatura
P17 · Modelos probabilísticos de difusión con eliminación de ruido	2020	La generación deja de ser un salto en la oscuridad: se aprende a deshacer, paso a paso, un proceso de ruido conocido.	notebook

Cada ficha explica el problema anterior, la matemática mínima, los límites y los errores de atribución más frecuentes. Para leerlas con método: cómo leer un paper de IA · anexos matemáticos.

📚 Bibliografía de apoyo

> Bloque generado por `python scripts/link_sources_to_classes.py`. Cada obra lleva su localizador verificado en `sources/bibliography.json`.

Los papers dicen **de dónde salió** el mecanismo. Estas obras lo **desarrollan** con el espacio que una clase no tiene: teoría completa, demostraciones y ejercicios.

Obra	Edición	Localizador	Papel en esta clase
Goodfellow, Ian, Bengio, Yoshua y Courville, Aaron — *Deep Learning*	2016	ISBN 9780262035613 · web de la obra	obra de referencia de la parte 07 · capítulos de modelos generativos

📝 Evaluación completa

❓ Preguntas

1. Define texto a imagen y condicionamiento sin usar una marca o framework como definición.
2. Explica la relación entre text-to-image, conditioning, guidance, prompt.
3. Ejecuta `lab.py` dos veces con la misma semilla. ¿Qué debe conservarse?
4. Identifica una afirmación permitida y una afirmación exagerada sobre el resultado.
5. Propón una prueba negativa o un caso límite.

🏆 Reto verificable

Amplía el resultado del laboratorio con una clave `student_extension` que incluya:

- el supuesto que estás probando;
- una medición o comprobación;
- la conclusión;
- una limitación.

✅ Criterio de aceptación

- [ ] `lab.py` termina con código 0.
- [ ] El resultado contiene `kind`, `seed`, `evidence` y `limitations`.
- [ ] La extensión no modifica el comportamiento de otras clases.
- [ ] La interpretación referencia datos impresos por el laboratorio.
- [ ] Se declara al menos un riesgo o condición de no uso.

---

092 — Control estructural y edición generativa

Parte: 07 — IA generativa para texto, imagen, audio, video y 3D
Nivel: avanzado · **Horas estimadas:** 6
Laboratorio: `generation` · **Estado:** `EXECUTABLE_CORE`

🎯 Propósito

Comprender **control estructural y edición generativa** dentro de la evolución de la inteligencia artificial, implementar un experimento mínimo verificable y distinguir qué parte constituye evidencia frente a una afirmación todavía no comprobada.

📚 Resultados de aprendizaje

Al finalizar podrás:

1. Explicar control estructural y edición generativa usando los conceptos `inpainting`, `control`, `pose`, `depth`.
2. Ejecutar el laboratorio con una semilla explícita y revisar su contrato JSON.
3. Identificar al menos un supuesto, una limitación y un riesgo de aplicación.
4. Comparar el enfoque con la etapa anterior de la ruta de aprendizaje.
5. Producir una evidencia reproducible y una conclusión que no exceda los datos.

🧩 Conceptos centrales

`inpainting`, `control`, `pose`, `depth`

🗺️ Ubicación en el mapa de la IA

El texto (clase 091) especifica *qué* generar, pero no *dónde ni cómo*: un prompt no puede fijar la pose exacta de una figura ni el trazado de un edificio. Esta clase cubre las técnicas que dan control espacial sobre un modelo de difusión ya entrenado —ControlNet, img2img e inpainting— sin reentrenarlo desde cero. Son las que convirtieron los generadores en herramientas de edición profesional y anticipan el control por condiciones múltiples en video y 3D.

📖 Fundamentos

🎛️ ControlNet: una copia entrenable del encoder

ControlNet (Zhang, Rao y Agrawala, 2023) añade condiciones espaciales densas —mapas de bordes (Canny), pose (OpenPose), profundidad, segmentación— a un modelo de difusión **congelado**. La arquitectura:

- Se **congela** la U-Net original (preserva todo lo aprendido con miles de millones de imágenes).
- Se crea una **copia entrenable** de los bloques del encoder de la U-Net, que recibe el latente ruidoso z_t más el mapa de condición c_f (bordes/pose/profundidad).

- La salida de cada bloque copiado se suma a la conexión skip correspondiente del decoder congelado, pasando por una **zero-convolution**: una convolución 1×1 con pesos W y sesgo b inicializados a **cero**.

```
salida_bloque = bloque_congelado(z_t)  +  Z(bloque_copiado(z_t + Z'(c_f)))
donde Z, Z' son convoluciones 1×1 con W = 0, b = 0 al inicio
```

Al comenzar el entrenamiento, $Z(\cdot) = 0$, así que la red se comporta **exactamente** como el modelo original: el control se incorpora de forma gradual sin destruir el conocimiento previo. Aunque la salida inicial es 0, el gradiente respecto a los pesos de la zero-convolution NO es cero ($\partial(W \cdot x)/\partial W = x \neq 0$), de modo que sí aprende desde el primer paso.

🖼️ img2img: la fuerza de ruido decide cuánto se conserva

img2img (formalizado como SDEdit, Meng et al., 2022) parte de una imagen real en vez de ruido puro. El parámetro **strength** $s \in [0, 1]$ controla el punto de entrada al proceso de difusión con T pasos totales:

```
1. z₀ = E(imagen_original)                  # encodificar al latente
2. t_inicio = round(s · T)                  # cuánto ruido añadir
3. z_{t_inicio} = √(ᾱ_t)·z₀ + √(1−ᾱ_t)·ε   # difusión hacia delante, un salto
4. denoising desde t_inicio hasta 0 con el prompt como condición
```

Con s pequeño, se añade poco ruido y quedan pocos pasos de denoising: el resultado conserva composición, colores y estructura del original. Con $s \to 1$, el latente es casi ruido puro y el original apenas influye. El costo también escala: se ejecutan $s \cdot T$ pasos de U-Net, no T.

🎭 Inpainting: regenerar solo dentro de la máscara

El inpainting recibe una imagen y una máscara binaria m (1 = regenerar, 0 = preservar). La variante sin reentrenamiento (RePaint, Lugmayr et al., 2022) fuerza en cada paso las zonas conocidas a su valor verdadero ruidificado:

```
en cada paso t:
    z_conocido  = difusión_forward(z₀_original, t)     # zona fuera de la máscara
    z_generado  = paso_de_denoising(z_t)               # predicción del modelo
    z_{t-1} = (1 − m) ⊙ z_conocido + m ⊙ z_generado    # componer con la máscara
```

El modelo solo "inventa" dentro de la máscara, pero cada paso ve el contexto real circundante, lo que garantiza coherencia en los bordes. Los modelos de inpainting dedicados (SD-inpainting) van más lejos: se afinan recibiendo la máscara y la imagen enmascarada como canales extra de entrada de la U-Net.

🧮 Ejemplo trabajado

La fuerza de img2img con T = 50. Calcula el paso inicial y los pasos ejecutados:

```
s = 0.2  →  t_inicio = round(0.2 · 50) = 10  →  10 pasos de denoising
s = 0.5  →  t_inicio = round(0.5 · 50) = 25  →  25 pasos
s = 0.8  →  t_inicio = round(0.8 · 50) = 40  →  40 pasos
s = 1.0  →  t_inicio = 50 (ruido casi puro)  →  50 pasos = texto-a-imagen normal
```

Con s = 0.2 el latente retiene $\sqrt{(\bar{\alpha}_{10})} \approx$ la mayor parte de la señal original: el modelo solo puede retocar texturas y estilo — la composición sobrevive. Con s = 0.8 queda poca señal original: el prompt domina y solo persisten rasgos globales (paleta, disposición aproximada de masas). Regla práctica: retoque de estilo s ≈ 0.3-0.5; reinterpretación fuerte s ≈ 0.7-0.9.

Gradiente de la zero-convolution en el primer paso. Sea la zero-conv y = W·x + b con W = 0, b = 0, y una feature de entrada x = 3.0. En el forward, y = 0: el ControlNet no altera al modelo congelado. En el backward, con gradiente entrante ∂L/∂y = g = 0.5:

```
∂L/∂W = g · x = 0.5 · 3.0 = 1.5   ≠ 0  → W se actualiza: W ← 0 − η·1.5
∂L/∂b = g     = 0.5               ≠ 0  → b se actualiza
∂L/∂x = g · W = 0.5 · 0   = 0          → el bloque copiado aún no recibe señal
```

Tras la primera actualización W ≠ 0, y a partir de ahí el gradiente también fluye hacia el interior de la copia entrenable: el control se "enciende" progresivamente.

📊 Propiedades y comparación

Técnica	Qué controla	¿Requiere entrenamiento?	Parámetros extra	Límite principal
Prompt (clase 091)	contenido semántico global	no	0	sin control espacial preciso
img2img (SDEdit)	composición global vía imagen inicial	no	0 (solo s)	trade-off rígido fidelidad/libertad
Inpainting (RePaint / SD-inpaint)	región enmascarada	no / fine-tuning ligero	0 / canales extra	coherencia global si la máscara es grande
ControlNet	estructura densa (pose, bordes, profundidad)	sí (copia del encoder, ~50 % params de la U-Net)	cientos de M	un modelo por tipo de condición
LoRA (parte 06)	estilo/sujeto aprendido	sí (rango bajo)	pocos M	no es control espacial por imagen

⚠️ Errores conceptuales frecuentes

1. **"ControlNet reentrena el modelo base."** No: la U-Net original queda congelada; solo se entrena la copia del encoder y las zero-convolutions. Por eso un ControlNet se acopla a cualquier checkpoint

compatible del mismo modelo base.

2. **"Si la zero-convolution sale 0, no puede aprender."** El gradiente respecto a sus *pesos* es proporcional a la *entrada* (no a la salida), así que es distinto de cero desde el primer paso — solo el gradiente hacia atrás a través de W es 0 inicialmente.
3. **"strength = 0.5 conserva el 50 % de los píxeles."** La fuerza fija el nivel de ruido de partida, no una mezcla lineal de píxeles: lo que se conserva es información de baja frecuencia (composición), no píxeles concretos.
4. **"El inpainting solo mira dentro de la máscara."** Al revés: en cada paso el modelo ve el contexto completo (la zona conocida ruidificada al nivel t); esa es la razón de que los bordes queden coherentes.
5. **"Más condiciones siempre dan mejor control."** Condiciones contradictorias (una pose que no cabe en el mapa de profundidad) degradan el resultado; los pesos de cada ControlNet deben balancearse y el prompt sigue mandando en lo semántico.

🚀 Del aprendizaje a la operación

Para un flujo de edición real faltan: preprocesadores robustos que extraigan la condición (detector de pose, estimador de profundidad) con sus propios modos de fallo, política de derechos cuando la imagen de partida no es del usuario, trazabilidad de qué se editó (máscaras y semillas registradas, procedencia C2PA), evaluación con usuarios de si el control es *suficiente* para la tarea, y límites de uso que impidan ediciones de personas reales sin consentimiento.

🧪 Laboratorio

```
python lab.py
```

El laboratorio llama a `ai_evolution.labs.run_lab("generation")`. Esta decisión evita una implementación divergente por clase: cada una conserva un entrypoint propio, pero los motores didácticos se prueban como una biblioteca común.

🔍 Evidencia esperada

- tipo de laboratorio y semilla;
- entradas o decisiones observables;
- resultado estructurado;
- lista `evidence` con hechos que pueden inspeccionarse;
- lista `limitations` que impide presentar la demo como producción.

📓 Notebooks

- 📓 `notebook.ipynb`: recorrido guiado con la materia resumida.
- ✍️ `notebook_student.ipynb`: ejercicios para resolver.
- ✅ `notebook_solution.ipynb`: solución de referencia explicada.

📝 Evaluación

Criterio	Peso
Comprensión conceptual	25 %
Ejecución reproducible	25 %
Interpretación basada en evidencia	25 %
Riesgos, límites y mejora propuesta	25 %

Consulta assessment.md para preguntas y criterio de aceptación.

⚠️ Errores comunes

Síntoma	Causa probable	Corrección
El código corre, pero no hay conclusión	Se confundió ejecución con aprendizaje	Explica qué demuestra y qué no demuestra
El resultado cambia sin explicación	No se registró semilla o configuración	Conserva semilla, versión y parámetros
Se promete uso real	Se extrapoló desde una demo educativa	Declara entorno, datos, límites y revisión humana
Se copia una métrica aislada	No existe baseline ni costo de error	Añade comparación y criterio de decisión

❓ Preguntas frecuentes

¿Debo usar una API comercial?
No. El núcleo funciona localmente. Las extensiones LIVE se documentan por separado.

¿El laboratorio representa una implementación industrial?
No por sí solo. Enseña el contrato y el patrón; producción exige integración, seguridad, observabilidad, pruebas y operación.

¿Dónde profundizo?
Revisa las especializaciones enlazadas en el README raíz y la ruta siguiente.

🔗 Referencias

- Zhang, L., Rao, A. y Agrawala, M. (2023). *Adding Conditional Control to Text-to-Image Diffusion Models* (ControlNet). arXiv:2302.05543 — uso: fuente primaria del mecanismo estudiado
- Meng, C. et al. (2022). *SDEdit: Guided Image Synthesis and Editing with Stochastic Differential Equations*. arXiv:2108.01073 — uso: fuente primaria del mecanismo estudiado
- Lugmayr, A. et al. (2022). *RePaint: Inpainting using Denoising Diffusion Probabilistic Models*. arXiv:2201.09865 — uso: fuente primaria del mecanismo estudiado
- Rombach, R. et al. (2022). *High-Resolution Image Synthesis with Latent Diffusion Models*. arXiv:2112.10752 — uso: fuente primaria del mecanismo estudiado
- Documentación de Hugging Face Diffusers: ControlNet — uso: referencia consultada en su fuente original

📜 Papers que fundamentan esta clase

> Bloque generado por `python scripts/link_papers_to_classes.py`. La fuente es `papers/catalog/papers.json`.

Paper	Año	Qué desbloqueó	Miniatura
P17 · Modelos probabilísticos de difusión con eliminación de ruido	2020	La generación deja de ser un salto en la oscuridad: se aprende a deshacer, paso a paso, un proceso de ruido conocido.	notebook

Cada ficha explica el problema anterior, la matemática mínima, los límites y los errores de atribución más frecuentes. Para leerlas con método: cómo leer un paper de IA · anexos matemáticos.

📚 Bibliografía de apoyo

> Bloque generado por `python scripts/link_sources_to_classes.py`. Cada obra lleva su localizador verificado en `sources/bibliography.json`.

Los papers dicen **de dónde salió** el mecanismo. Estas obras lo **desarrollan** con el espacio que una clase no tiene: teoría completa, demostraciones y ejercicios.

Obra	Edición	Localizador	Papel en esta clase
Goodfellow, Ian, Bengio, Yoshua y Courville, Aaron — *Deep Learning*	2016	ISBN 9780262035613 · web de la obra	obra de referencia de la parte 07 · capítulos de modelos generativos

📝 Evaluación completa

❓ Preguntas

1. Define control estructural y edición generativa sin usar una marca o framework como definición.
2. Explica la relación entre inpainting, control, pose, depth.
3. Ejecuta `lab.py` dos veces con la misma semilla. ¿Qué debe conservarse?
4. Identifica una afirmación permitida y una afirmación exagerada sobre el resultado.
5. Propón una prueba negativa o un caso límite.

🏆 Reto verificable

Amplía el resultado del laboratorio con una clave `student_extension` que incluya:

- el supuesto que estás probando;
- una medición o comprobación;

- la conclusión;
- una limitación.

✅ Criterio de aceptación

- [ ] `lab.py` termina con código 0.
- [ ] El resultado contiene `kind`, `seed`, `evidence` y `limitations`.
- [ ] La extensión no modifica el comportamiento de otras clases.
- [ ] La interpretación referencia datos impresos por el laboratorio.
- [ ] Se declara al menos un riesgo o condición de no uso.

---

093 — Generación musical y de audio

Parte: 07 — IA generativa para texto, imagen, audio, video y 3D
Nivel: avanzado · **Horas estimadas:** 6
Laboratorio: `generation` · **Estado:** `EXECUTABLE_CORE`

🎯 Propósito

Comprender **generación musical y de audio** dentro de la evolución de la inteligencia artificial, implementar un experimento mínimo verificable y distinguir qué parte constituye evidencia frente a una afirmación todavía no comprobada.

📚 Resultados de aprendizaje

Al finalizar podrás:

1. Explicar generación musical y de audio usando los conceptos `audio`, `música`, `codecs`, `difusión`.
2. Ejecutar el laboratorio con una semilla explícita y revisar su contrato JSON.
3. Identificar al menos un supuesto, una limitación y un riesgo de aplicación.
4. Comparar el enfoque con la etapa anterior de la ruta de aprendizaje.
5. Producir una evidencia reproducible y una conclusión que no exceda los datos.

🧩 Conceptos centrales

`audio`, `música`, `codecs`, `difusión`

🗺️ Ubicación en el mapa de la IA

El audio plantea un problema que la imagen no tiene: la escala temporal. Un segundo de audio crudo son 24 000-48 000 muestras, así que generar un minuto muestra a muestra (WaveNet, 2016) era prohibitivo. La solución moderna —códecs neuronales que comprimen la onda a tokens discretos (SoundStream, EnCodec) sobre los que un transformer autorregresivo genera como si fuera texto (AudioLM, MusicGen)— traslada al audio las técnicas de las partes 05-06, y compite con la difusión sobre espectrogramas. La clase 094 (síntesis de voz) reutiliza estas mismas representaciones.

📖 Fundamentos

🎵 Tres representaciones del audio

- **Forma de onda**: la señal cruda x(t), una amplitud por muestra. PCM sin comprimir a 24 kHz y 16 bits son 384 000 bits/s. Máxima fidelidad, secuencias larguísimas.
- **Espectrograma mel**: magnitud de la STFT proyectada a ~80-128 bandas mel (escala perceptual de frecuencia), típicamente ~100 frames/s. Es una "imagen" tiempo×frecuencia: sirve para difusión estilo imagen, pero descarta la fase — hace falta un **vocoder** (HiFi-GAN, difusión) para volver a la onda.

- **Tokens neuronales**: un códec neuronal (encoder convolucional + cuantización + decoder) convierte la onda en una secuencia corta de índices discretos de vocabulario finito. Es la representación nativa para transformers.

🧊 Códecs neuronales y cuantización vectorial residual (RVQ)

SoundStream (Zeghidour et al., 2021) y EnCodec (Défossez et al., 2022) comprimen así: el encoder reduce 24 000 muestras/s a ~75 vectores latentes/s (stride total 320). Cada vector continuo se discretiza con **RVQ**: una cascada de Q codebooks donde cada uno cuantiza el *residuo* que dejó el anterior:

```
r₁ = z                      # vector latente del frame
para q = 1 … Q:
    kq = argmin_k ‖rq − C_q[k]‖    # entrada más cercana del codebook q
    r_{q+1} = rq − C_q[kq]         # el residuo pasa al siguiente codebook
z ≈ C_1[k1] + C_2[k2] + … + C_Q[kQ]
```

Cada frame queda representado por Q índices (uno por codebook). Con codebooks de K entradas, cada índice cuesta $\log_2(K)$ bits. La primera capa captura lo grueso; las siguientes refinan detalles: se puede truncar a menos codebooks para bajar el bitrate a costa de calidad (bitrate escalable). El entrenamiento combina pérdida de reconstrucción espectral, pérdida adversarial (discriminadores sobre STFT) y compromiso de cuantización.

🔁 Dos familias de modelos generativos de audio

Autorregresivos sobre tokens: AudioLM y MusicGen tratan los índices RVQ como un "idioma". MusicGen (Copet et al., 2023) condiciona por descripción de texto y genera los Q streams de codebooks con un solo transformer usando un **patrón de retardo** (delay pattern): en el paso t predice el codebook 1 del frame t, el 2 del frame t−1, etc., evitando Q pasadas separadas. Ventajas: continuación natural de audio, streaming. Costo: generación secuencial, un token cada vez.

Difusión sobre espectrogramas: aplicar el pipeline de la clase 090-088 tratando el espectrograma mel como imagen (p. ej. Riffusion sobre Stable Diffusion) y reconstruir la onda con un vocoder. Ventajas: hereda toda la maquinaria de imagen (CFG, img2img sobre música). Costo: la fase se pierde, la duración es fija por "lienzo", y la coherencia musical de largo plazo es más difícil.

```
Pipeline autorregresivo (MusicGen):
texto → encoder T5 → transformer AR sobre tokens RVQ → decoder EnCodec → onda
```

🧮 Ejemplo trabajado

Bitrate de un códec neuronal, a mano. Audio a 24 kHz, códec a 75 frames/s con Q = 8 codebooks de K = 1024 entradas:

```
bits por índice   = log₂(1024) = 10
bits por frame    = 8 codebooks · 10 bits = 80
bitrate           = 75 frames/s · 80 bits = 6 000 bits/s = 6 kbps

PCM sin comprimir = 24 000 muestras/s · 16 bits = 384 000 bits/s = 384 kbps
factor de compresión = 384 000 / 6 000 = 64×
```

Longitud de secuencia para el transformer. 30 segundos de música:

```
frames   = 30 s · 75 = 2 250
tokens   = 2 250 · 8 codebooks = 18 000 tokens (si se aplanara todo)
con delay pattern (MusicGen): ~2 250 + 7 pasos del transformer, con 8 cabezas
                              de salida en paralelo por paso
```

Aplanar los 8 codebooks daría 18 000 posiciones — inviable con atención cuadrática para piezas largas. El patrón de retardo reduce la secuencia efectiva a ~2 257 pasos. Comparación: modelar la onda cruda muestra a muestra serían 30 · 24 000 = 720 000 pasos autorregresivos, 320 veces más. La compresión del códec es lo que hace posible la generación musical con transformers.

📊 Propiedades y comparación

Enfoque	Representación	Secuencia (30 s)	Generación	Trade-off principal
WaveNet (2016)	onda cruda	720 000 muestras	AR muestra a muestra	fidelidad alta, lentitud extrema
Difusión sobre mel + vocoder	espectrograma ~100 fr/s	lienzo fijo ~3 000 frames	paralela (T pasos)	pierde fase; duración rígida
AR sobre tokens de códec (MusicGen, AudioLM)	tokens RVQ 75 fr/s × Q	~2 250 pasos (delay)	AR token a token	coherencia larga limitada por contexto
Códec solo (EnCodec/SoundStream)	tokens RVQ	—	no genera: comprime	calidad acotada por el bitrate elegido

⚠️ Errores conceptuales frecuentes

1. **"El códec neuronal es como MP3."** MP3 usa psicoacústica y transformadas fijas; un códec neuronal aprende la compresión y produce **tokens discretos de un vocabulario**, que es justo lo que

un transformer necesita como entrada. MP3 no da una secuencia modelable de ese modo.
2. **"Más codebooks = mejor música generada."** Más codebooks mejoran la *reconstrucción* del códec, pero multiplican los tokens a predecir: el modelo generativo puede empeorar porque la secuencia se alarga y los codebooks finos son casi ruido difícil de predecir.
3. **"El espectrograma es invertible."** El espectrograma de magnitud descarta la fase; recuperar la onda exige un vocoder o Griffin-Lim, y esa etapa introduce sus propios artefactos.
4. **"La generación AR de audio entiende estructura musical."** La coherencia surge de la ventana de contexto (segundos, no minutos): forma sonata o estribillos que regresan tras 2 minutos exceden el contexto típico y requieren condicionamiento jerárquico (como los tokens semánticos de AudioLM).
5. **"Se puede evaluar solo con métricas."** FAD (Fréchet Audio Distance) y CLAP score se correlacionan débilmente con juicio musical humano; la evaluación seria incluye pruebas de escucha ciegas.

🚀 Del aprendizaje a la operación

Un servicio real de generación musical añade: derechos sobre los datos de entrenamiento (catálogos con licencia, no scraping), detección de imitación de artistas concretos y filtros de voz clonada, marcas de agua acústicas y procedencia, latencia de streaming (generar más rápido que tiempo real), y controles musicales útiles (tonalidad, tempo, estructura) que el condicionamiento por texto libre no garantiza.

🧪 Laboratorio

```
python lab.py
```

El laboratorio llama a `ai_evolution.labs.run_lab("generation")`. Esta decisión evita una implementación divergente por clase: cada una conserva un entrypoint propio, pero los motores didácticos se prueban como una biblioteca común.

🔍 Evidencia esperada

- tipo de laboratorio y semilla;
- entradas o decisiones observables;
- resultado estructurado;
- lista `evidence` con hechos que pueden inspeccionarse;
- lista `limitations` que impide presentar la demo como producción.

📓 Notebooks

- 📓 `notebook.ipynb`: recorrido guiado con la materia resumida.
- ✍️ `notebook_student.ipynb`: ejercicios para resolver.
- ✅ `notebook_solution.ipynb`: solución de referencia explicada.

📝 Evaluación

Criterio	Peso
Comprensión conceptual	25 %
Ejecución reproducible	25 %
Interpretación basada en evidencia	25 %
Riesgos, límites y mejora propuesta	25 %

Consulta assessment.md para preguntas y criterio de aceptación.

⚠️ Errores comunes

Síntoma	Causa probable	Corrección
El código corre, pero no hay conclusión	Se confundió ejecución con aprendizaje	Explica qué demuestra y qué no demuestra
El resultado cambia sin explicación	No se registró semilla o configuración	Conserva semilla, versión y parámetros
Se promete uso real	Se extrapoló desde una demo educativa	Declara entorno, datos, límites y revisión humana
Se copia una métrica aislada	No existe baseline ni costo de error	Añade comparación y criterio de decisión

❓ Preguntas frecuentes

¿Debo usar una API comercial?
No. El núcleo funciona localmente. Las extensiones LIVE se documentan por separado.

¿El laboratorio representa una implementación industrial?
No por sí solo. Enseña el contrato y el patrón; producción exige integración, seguridad, observabilidad, pruebas y operación.

¿Dónde profundizo?
Revisa las especializaciones enlazadas en el README raíz y la ruta siguiente.

🔗 Referencias

- Copet, J. et al. (2023). *Simple and Controllable Music Generation* (MusicGen). arXiv:2306.05284 — uso: fuente primaria del mecanismo estudiado
- Défossez, A., Copet, J., Synnaeve, G. y Adi, Y. (2022). *High Fidelity Neural Audio Compression* (EnCodec). arXiv:2210.13438 — uso: fuente primaria del mecanismo estudiado
- Zeghidour, N. et al. (2021). *SoundStream: An End-to-End Neural Audio Codec*. arXiv:2107.03312 — uso: fuente primaria del mecanismo estudiado
- Borsos, Z. et al. (2022). *AudioLM: a Language Modeling Approach to Audio Generation*. arXiv:2209.03143 — uso: fuente primaria del mecanismo estudiado
- van den Oord, A. et al. (2016). *WaveNet: A Generative Model for Raw Audio*. arXiv:1609.03499 — uso: fuente primaria del mecanismo estudiado

📜 Papers que fundamentan esta clase

> Bloque generado por `python scripts/link_papers_to_classes.py`. La fuente es `papers/catalog/papers.json`.

Paper	Año	Qué desbloqueó	Miniatura
P127 · Jukebox: un modelo generativo de música	2020	Genera canciones con voz cantada reconocible modelando códigos discretos en tres escalas temporales, en vez de la forma de onda directamente.	notebook
P129 · MusicLM: generar música a partir de texto	2023	Genera música coherente de varios minutos desde una descripción en lenguaje natural, y publica MusicCaps para que la tarea se pueda evaluar.	notebook

Cada ficha explica el problema anterior, la matemática mínima, los límites y los errores de atribución más frecuentes. Para leerlas con método: cómo leer un paper de IA · anexos matemáticos.

📚 Bibliografía de apoyo

> Bloque generado por `python scripts/link_sources_to_classes.py`. Cada obra lleva su localizador verificado en `sources/bibliography.json`.

Los papers dicen **de dónde salió** el mecanismo. Estas obras lo **desarrollan** con el espacio que una clase no tiene: teoría completa, demostraciones y ejercicios.

Obra	Edición	Localizador	Papel en esta clase
Goodfellow, Ian, Bengio, Yoshua y Courville, Aaron — *Deep Learning*	2016	ISBN 9780262035613 · web de la obra	obra de referencia de la parte 07 · capítulos de modelos generativos

📝 Evaluación completa

❓ Preguntas

1. Define generación musical y de audio sin usar una marca o framework como definición.
2. Explica la relación entre audio, música, codecs, difusión.
3. Ejecuta `lab.py` dos veces con la misma semilla. ¿Qué debe conservarse?
4. Identifica una afirmación permitida y una afirmación exagerada sobre el resultado.
5. Propón una prueba negativa o un caso límite.

🏆 Reto verificable

Amplía el resultado del laboratorio con una clave `student_extension` que incluya:

- el supuesto que estás probando;
- una medición o comprobación;
- la conclusión;
- una limitación.

✅ Criterio de aceptación

- [ ] `lab.py` termina con código 0.
- [ ] El resultado contiene `kind`, `seed`, `evidence` y `limitations`.
- [ ] La extensión no modifica el comportamiento de otras clases.
- [ ] La interpretación referencia datos impresos por el laboratorio.
- [ ] Se declara al menos un riesgo o condición de no uso.

---

094 — Síntesis de voz y derechos de identidad

Parte: 07 — IA generativa para texto, imagen, audio, video y 3D
Nivel: avanzado · **Horas estimadas:** 6
Laboratorio: `safety` · **Estado:** `EXECUTABLE_CORE`

🎯 Propósito

Comprender **síntesis de voz y derechos de identidad** dentro de la evolución de la inteligencia artificial, implementar un experimento mínimo verificable y distinguir qué parte constituye evidencia frente a una afirmación todavía no comprobada.

📚 Resultados de aprendizaje

Al finalizar podrás:

1. Explicar síntesis de voz y derechos de identidad usando los conceptos `TTS`, `identidad`, `consentimiento`, `abuso`.
2. Ejecutar el laboratorio con una semilla explícita y revisar su contrato JSON.
3. Identificar al menos un supuesto, una limitación y un riesgo de aplicación.
4. Comparar el enfoque con la etapa anterior de la ruta de aprendizaje.
5. Producir una evidencia reproducible y una conclusión que no exceda los datos.

🧩 Conceptos centrales

`TTS`, `identidad`, `consentimiento`, `abuso`

🗺️ Ubicación en el mapa de la IA

La síntesis de voz (TTS, *text-to-speech*) pasó de sistemas concatenativos —pegar fragmentos grabados— a redes neuronales de extremo a extremo: Tacotron 2 (2017) demostró que un modelo secuencia-a-secuencia más un vocoder neuronal alcanza naturalidad cercana a la humana. Hereda directamente la generación de audio de la clase anterior (093) y añade un problema que ningún otro medio plantea con la misma crudeza: la voz **es** un identificador biométrico de una persona concreta. Por eso esta clase une arquitectura técnica (mel-espectrogramas, vocoders, clonación zero-shot) con derechos de identidad y consentimiento, y prepara la discusión de procedencia y autenticidad de la clase 098.

📖 Fundamentos

🔊 El pipeline TTS neuronal: texto → fonemas → mel → onda

Un sistema TTS moderno se descompone en tres etapas con representaciones intermedias bien definidas:

```
texto  ──normalización──▶  fonemas / grafemas
fonemas ──modelo acústico──▶ mel-espectrograma  (T frames × 80 bandas mel)
mel    ──vocoder──▶  forma de onda  (samples a 22 050 o 24 000 Hz)
```

1. **Front-end lingüístico**: normaliza el texto ("Dr." → "doctor", "1984" → año o número según contexto) y lo convierte en fonemas. Los errores aquí producen pronunciaciones absurdas aunque el resto del sistema sea perfecto.
2. **Modelo acústico**: predice un **mel-espectrograma** — una matriz tiempo × frecuencia donde cada columna (frame) resume una ventana corta de audio (típicamente 50 ms con salto o *hop* de 256 muestras) y cada fila es una banda de frecuencia en escala mel, perceptualmente uniforme. Es una representación compacta: 80 números por frame en lugar de 256 muestras de onda.
3. **Vocoder**: convierte el mel-espectrograma en forma de onda. El mel descarta la fase, así que el vocoder debe *inventarla* de forma plausible.

🧬 Modelos acústicos: autorregresivo vs paralelo

Tacotron 2 (Shen et al., 2018) es autorregresivo: un encoder procesa los caracteres, un decoder con atención genera el mel frame a frame, condicionado en los frames previos. Ventaja: prosodia natural aprendida de los datos. Desventajas: velocidad lineal en la longitud, y fallos de atención (palabras repetidas u omitidas) cuando la alineación texto-audio se pierde.

FastSpeech 2 (Ren et al., 2020) es no autorregresivo: predice explícitamente la **duración de cada fonema** (con un *duration predictor* entrenado con alineaciones), expande la secuencia de fonemas a la longitud del mel y genera todos los frames en paralelo, añadiendo predictores de tono (F0) y energía como condicionamiento. Elimina los fallos de atención y acelera la inferencia en más de un orden de magnitud, a costa de una prosodia algo más plana si los predictores son pobres.

🎛️ Vocoders: WaveNet vs HiFi-GAN

- **WaveNet** (van den Oord et al., 2016): autorregresivo a nivel de muestra — genera las 22 050 muestras de cada segundo *una por una*, cada una condicionada en las anteriores mediante convoluciones dilatadas. Calidad excelente, pero inferencia extremadamente lenta (miles de pasos secuenciales por segundo de audio).
- **HiFi-GAN** (Kong et al., 2020): generador convolucional entrenado adversarialmente con discriminadores multi-escala y multi-período que vigilan la estructura periódica de la voz. Genera toda la onda en paralelo: cientos de veces más rápido que tiempo real en GPU, con calidad comparable.

La métrica operativa es el **factor de tiempo real** (RTF): segundos de cómputo por segundo de audio generado. RTF < 1 permite aplicaciones interactivas; un vocoder autorregresivo puro suele tener RTF ≫ 1 sin optimizaciones dedicadas.

🧬 Clonación de voz zero-shot y speaker embeddings

SV2TTS (Jia et al., 2018) demostró que se puede clonar una voz **sin reentrenar**:

```
audio de referencia (segundos) ──encoder de hablante──▶ speaker embedding e ∈ R^256
TTS(texto, e) ──▶ voz sintética con el timbre del hablante de referencia
```

El encoder de hablante se entrena en una tarea distinta (verificación de locutor, con miles de hablantes) para que su embedding capture el timbre y no el contenido. El TTS, condicionado en ese embedding, generaliza a hablantes **nunca vistos**: basta un clip de pocos segundos. Esta es exactamente la capacidad que habilita tanto usos legítimos (voces protésicas para pacientes de ELA, doblaje consentido) como abusos (suplantación telefónica, fraude a familiares, deepfakes de figuras públicas).

⚖️ Derechos de identidad y consentimiento

La voz clonada plantea tres capas de problema jurídico-técnico:

1. **Consentimiento**: ¿autorizó la persona el uso de su voz para entrenar o clonar? El consentimiento debe ser específico (para qué usos), revocable y verificable.
2. **Suplantación**: usar la voz clonada para hacerse pasar por la persona (fraude, ingeniería social). Es delito en la mayoría de jurisdicciones con independencia de cómo se generó el audio.
3. **Procedencia**: sin marcas técnicas (watermarking de audio, credenciales C2PA), un oyente no puede distinguir voz real de sintética; la defensa se desplaza a la verificación de origen, no del contenido (clase 098).

🧮 Ejemplo trabajado

Dimensiones de un mel-espectrograma. Audio de 3 s muestreado a 22 050 Hz, con hop de 256 muestras y 80 bandas mel:

```
muestras totales:  3 × 22 050 = 66 150
frames:            66 150 / 256 ≈ 258.4  →  ~259 frames (con padding de borde)
mel-espectrograma: 259 frames × 80 bandas = 20 720 valores
forma de onda:     66 150 valores
compresión:        66 150 / 20 720 ≈ 3.2× (y además sin fase)
```

Factor de tiempo real (RTF) de vocoders. Supongamos que un vocoder autorregresivo genera 500 muestras por segundo de cómputo y uno paralelo procesa el clip entero en 0.05 s de GPU:

```
autorregresivo: 66 150 muestras / 500 muestras·s⁻¹ = 132.3 s de cómputo
                RTF = 132.3 / 3 = 44.1   (44× más lento que tiempo real)
paralelo:       RTF = 0.05 / 3 ≈ 0.017  (60× más rápido que tiempo real)
razón:          44.1 / 0.017 ≈ 2 600×
```

La diferencia no es una constante de implementación: es estructural. El autorregresivo tiene 66 150 pasos secuenciales irreducibles; el paralelo, unas decenas de capas.

📊 Propiedades y comparación

Sistema	Tipo	Velocidad (RTF típico)	Riesgo característico	Control de prosodia
Tacotron 2 + WaveNet	autorregresivo × 2	$\gg 1$ (lento)	fallos de atención (repeticiones/omisiones)	implícito (aprendido)
FastSpeech 2 + HiFi-GAN	paralelo × 2	$\ll 1$ (interactivo)	prosodia plana si los predictores fallan	explícito (duración, F0, energía)
Clonación zero-shot (SV2TTS)	condicionado por embedding	según backbone	suplantación de identidad	hereda del backbone
Concatenativo clásico	unidades grabadas	$\ll 1$	juntas audibles, sin flexibilidad	casi nulo

⚠️ Errores conceptuales frecuentes

1. **"El mel-espectrograma contiene toda la información del audio."** No: descarta la fase. Por eso el vocoder es un modelo generativo (debe inventar una fase plausible) y no un simple inversor de la transformada.
2. **"Más rápido implica peor calidad."** HiFi-GAN paralelo iguala en escucha ciega a WaveNet autorregresivo siendo órdenes de magnitud más rápido; el cuello de botella histórico era estructural (secuencialidad), no de capacidad.
3. **"Clonar una voz requiere horas de grabación del objetivo."** La clonación zero-shot con speaker embeddings funciona con segundos de audio; asumir lo contrario subestima gravemente el riesgo de suplantación.
4. **"Si el audio suena natural, es indetectable."** Naturalidad perceptual y detectabilidad forense son ejes distintos: artefactos espectrales, marcas de agua y metadatos de procedencia pueden delatar audio que el oído acepta.
5. **"El consentimiento para grabar equivale al consentimiento para clonar."** Son permisos distintos: ceder una grabación para un podcast no autoriza entrenar un clon que diga frases nuevas con tu timbre.

🚀 Del aprendizaje a la operación

Entre este núcleo educativo y un producto real de TTS faltan: un front-end lingüístico robusto para el idioma objetivo (normalización de números, siglas y extranjerismos), datos de estudio con licencia y consentimiento documentado por hablante, streaming de baja latencia (generar audio antes de terminar la frase), watermarking y credenciales de procedencia integrados en el pipeline de salida, y un proceso de verificación de identidad antes de habilitar clonación de voz de terceros.

🧪 Laboratorio

```
python lab.py
```

El laboratorio llama a `ai_evolution.labs.run_lab("safety")`. Esta decisión evita una implementación divergente por clase: cada una conserva un entrypoint propio, pero los motores didácticos se prueban como una biblioteca común.

🔍 Evidencia esperada

- tipo de laboratorio y semilla;
- entradas o decisiones observables;
- resultado estructurado;
- lista `evidence` con hechos que pueden inspeccionarse;
- lista `limitations` que impide presentar la demo como producción.

📓 Notebooks

- 📓 `notebook.ipynb`: recorrido guiado con la materia resumida.
- ✍️ `notebook_student.ipynb`: ejercicios para resolver.
- ✅ `notebook_solution.ipynb`: solución de referencia explicada.

📝 Evaluación

Criterio	Peso
Comprensión conceptual	25 %
Ejecución reproducible	25 %
Interpretación basada en evidencia	25 %
Riesgos, límites y mejora propuesta	25 %

Consulta assessment.md para preguntas y criterio de aceptación.

⚠️ Errores comunes

Síntoma	Causa probable	Corrección
El código corre, pero no hay conclusión	Se confundió ejecución con aprendizaje	Explica qué demuestra y qué no demuestra
El resultado cambia sin explicación	No se registró semilla o configuración	Conserva semilla, versión y parámetros
Se promete uso real	Se extrapoló desde una demo educativa	Declara entorno, datos, límites y revisión humana
Se copia una métrica aislada	No existe baseline ni costo de error	Añade comparación y criterio de decisión

❓ Preguntas frecuentes

¿Debo usar una API comercial?
No. El núcleo funciona localmente. Las extensiones LIVE se documentan por separado.

¿El laboratorio representa una implementación industrial?
No por sí solo. Enseña el contrato y el patrón; producción exige integración, seguridad, observabilidad, pruebas y operación.

¿Dónde profundizo?
Revisa las especializaciones enlazadas en el README raíz y la ruta siguiente.

🔗 Referencias

- Shen, J. et al. (2018). *Natural TTS Synthesis by Conditioning WaveNet on Mel Spectrogram Predictions* (Tacotron 2). arXiv:1712.05884 — uso: fuente primaria del mecanismo estudiado
- Ren, Y. et al. (2020). *FastSpeech 2: Fast and High-Quality End-to-End Text to Speech*. arXiv:2006.04558 — uso: fuente primaria del mecanismo estudiado
- Kong, J., Kim, J. y Bae, J. (2020). *HiFi-GAN: Generative Adversarial Networks for Efficient and High Fidelity Speech Synthesis*. arXiv:2010.05646 — uso: fuente primaria del mecanismo estudiado
- van den Oord, A. et al. (2016). *WaveNet: A Generative Model for Raw Audio*. arXiv:1609.03499 — uso: fuente primaria del mecanismo estudiado
- Jia, Y. et al. (2018). *Transfer Learning from Speaker Verification to Multispeaker Text-To-Speech Synthesis* (SV2TTS). arXiv:1806.04558 — uso: fuente primaria del mecanismo estudiado
- C2PA. *Content Credentials: C2PA Technical Specification*. c2pa.org/specifications — uso: marco normativo de referencia

---

📜 Papers que fundamentan esta clase

> Bloque generado por `python scripts/link_papers_to_classes.py`. La fuente es `papers/catalog/papers.json`.

Paper	Año	Qué desbloqueó	Miniatura
P130 · Los modelos de lenguaje sobre códecs neuronales sintetizan voz sin ejemplos previos	2023	Convierte la síntesis de voz en modelado de lenguaje sobre códigos de audio, y clona una voz con tres segundos de muestra sin entrenar nada.	notebook

Cada ficha explica el problema anterior, la matemática mínima, los límites y los errores de atribución más frecuentes. Para leerlas con método: cómo leer un paper de IA · anexos matemáticos.

📚 Bibliografía de apoyo

> Bloque generado por `python scripts/link_sources_to_classes.py`. Cada obra lleva su localizador verificado en `sources/bibliography.json`.

Los papers dicen **de dónde salió** el mecanismo. Estas obras lo **desarrollan** con el espacio que una clase no tiene: teoría completa, demostraciones y ejercicios.

Obra	Edición	Localizador	Papel en esta clase
Goodfellow, Ian, Bengio, Yoshua y Courville, Aaron — *Deep Learning*	2016	ISBN 9780262035613 · web de la obra	obra de referencia de la parte 07 · capítulos de modelos generativos

Normas y documentación oficial que aplica esta clase: c2pa.org/specifications

📝 Evaluación completa

❓ Preguntas

1. Define síntesis de voz y derechos de identidad sin usar una marca o framework como definición.
2. Explica la relación entre TTS, identidad, consentimiento, abuso.
3. Ejecuta `lab.py` dos veces con la misma semilla. ¿Qué debe conservarse?
4. Identifica una afirmación permitida y una afirmación exagerada sobre el resultado.
5. Propón una prueba negativa o un caso límite.

🏆 Reto verificable

Amplía el resultado del laboratorio con una clave `student_extension` que incluya:

- el supuesto que estás probando;
- una medición o comprobación;
- la conclusión;
- una limitación.

✅ Criterio de aceptación

- [ ] `lab.py` termina con código 0.
- [ ] El resultado contiene `kind`, `seed`, `evidence` y `limitations`.
- [ ] La extensión no modifica el comportamiento de otras clases.
- [ ] La interpretación referencia datos impresos por el laboratorio.
- [ ] Se declara al menos un riesgo o condición de no uso.

---

095 — Generación y edición de video

Parte: 07 — IA generativa para texto, imagen, audio, video y 3D
Nivel: avanzado · **Horas estimadas:** 6
Laboratorio: `generation` · **Estado:** `EXECUTABLE_CORE`

🎯 Propósito

Comprender **generación y edición de video** dentro de la evolución de la inteligencia artificial, implementar un experimento mínimo verificable y distinguir qué parte constituye evidencia frente a una afirmación todavía no comprobada.

📚 Resultados de aprendizaje

Al finalizar podrás:

1. Explicar generación y edición de video usando los conceptos `video`, `temporalidad`, `consistencia`, `edición`.
2. Ejecutar el laboratorio con una semilla explícita y revisar su contrato JSON.
3. Identificar al menos un supuesto, una limitación y un riesgo de aplicación.
4. Comparar el enfoque con la etapa anterior de la ruta de aprendizaje.
5. Producir una evidencia reproducible y una conclusión que no exceda los datos.

🧩 Conceptos centrales

`video`, `temporalidad`, `consistencia`, `edición`

🗺️ Ubicación en el mapa de la IA

La generación de video hereda toda la maquinaria de los modelos de difusión de imagen (clases 090-091) y añade la dimensión que las imágenes no tienen: el **tiempo**. Un video no es una lista de imágenes independientes — exige consistencia de identidad, iluminación y física entre frames. Video Diffusion Models (Ho et al., 2022) extendió la difusión al eje temporal; Make-A-Video y Stable Video Diffusion la llevaron a escala con difusión latente. Esta clase cierra la progresión imagen → audio → video y prepara la generación 3D (clase 096), donde la consistencia deja de ser temporal y pasa a ser geométrica.

📖 Fundamentos

🎞️ El problema: una dimensión más lo cambia todo

Un video de F frames de H×W píxeles es un tensor `F × H × W × 3`. Tratarlo como F imágenes independientes produce *parpadeo*: cada frame es plausible por separado, pero la identidad del sujeto, la textura y la iluminación cambian entre frames. El modelo debe aprender **dependencias temporales**: qué se mueve, qué permanece y cómo.

Dos costos explotan con la dimensión temporal:

- **Memoria/cómputo**: la atención completa sobre todos los tokens espacio-temporales es $O(n^2)$ con $n = F \cdot h \cdot w$ tokens (h, w en el espacio latente).
- **Datos**: el video etiquetado escasea; Make-A-Video mostró que se puede aprender apariencia de pares texto-imagen y movimiento de video **sin texto**.

🧊 Difusión de video latente

Igual que en imagen (Stable Diffusion), se trabaja en un espacio latente comprimido:

```
video F×H×W×3 ──VAE──▶ latentes F×h×w×c   (p. ej. h = H/8, w = W/8)
difusión: aprender a revertir el ruido sobre el tensor latente completo
decodificación: VAE⁻¹ frame a frame (o con decoder temporal para evitar parpadeo)
```

Stable Video Diffusion (2023) parte de un modelo de imagen preentrenado, **infla** la U-Net insertando capas temporales (convoluciones 1D y atención sobre el eje F) y la ajusta en tres etapas: preentrenamiento de imagen, preentrenamiento de video con datos masivos filtrados, y afinado de alta calidad. La lección empírica: la curación del dataset de video pesa tanto como la arquitectura.

🧠 Atención espacio-temporal: completa vs factorizada

Con F frames de h×w latentes hay $n = F \cdot h \cdot w$ tokens. Opciones:

```
1. Atención completa 3D:   cada token atiende a todos → O((F·h·w)²)
   máxima expresividad, costo prohibitivo al crecer F o la resolución.

2. Atención factorizada:   espacial + temporal por separado
   - espacial:  F bloques independientes de h·w tokens → F·(h·w)²
   - temporal:  h·w posiciones que atienden a lo largo de F frames → h·w·F²
   costo total F·(h·w)² + h·w·F² ≪ (F·h·w)²
```

La factorizada es el estándar en 3D U-Nets y en muchos DiT de video: alterna bloques espaciales (consistencia dentro del frame) y temporales (consistencia entre frames). El precio: ningún token ve *directamente* otro token en otro frame y otra posición — la información viaja en dos saltos, lo que puede degradar movimientos complejos.

✂️ Edición: video2video y propagación de ediciones

- **Video2video**: en lugar de partir de ruido puro, se parte del video original ruidificado hasta un nivel intermedio y se elimina el ruido condicionando en la nueva instrucción (cambiar estilo, sustituir un objeto). El nivel de ruido controla el trade-off fidelidad ↔ libertad de edición.
- **Propagación de ediciones**: se edita un frame clave (con herramientas de imagen, más maduras) y la edición se propaga al resto usando correspondencias temporales (flujo óptico o atención cruzada entre frames). Falla cuando hay oclusiones o cambios de plano: la correspondencia deja de existir.
- **Inversión**: para editar fielmente hace falta recuperar el ruido que "genera" el video original (inversión DDIM); los errores de inversión se acumulan por frame y son la fuente típica de deriva de identidad en ediciones largas.

📏 Métricas y evaluación

No hay una métrica única de calidad de video. Se combinan: FVD (distancia de Fréchet sobre features de video, sensible a la dinámica), consistencia temporal (similitud entre frames alineados por flujo), fidelidad al prompt (CLIP score por frame) y evaluación humana. Un modelo puede ganar en frames individuales y perder en FVD por movimiento incoherente: las métricas por frame no capturan la temporalidad.

🧮 Ejemplo trabajado

Costo de atención completa vs factorizada. 16 frames con latentes de 32×32 (h·w = 1024 tokens por frame):

```
n = 16 × 1024 = 16 384 tokens espacio-temporales

Atención completa:    n² = 16 384² = 268 435 456 ≈ 2.7 × 10⁸ pares

Factorizada:
  espacial:  16 · (1024²) = 16 · 1 048 576 = 16 777 216
  temporal:  1024 · (16²) = 1024 · 256    =    262 144
  total:     16 777 216 + 262 144         = 17 039 360 ≈ 1.7 × 10⁷ pares

Razón: 268 435 456 / 17 039 360 ≈ 15.8×
```

La factorización ahorra ~16× aquí, y el ahorro **crece** con la longitud: con 64 frames, la completa escala ×16 (n^2 con $n \times 4$) mientras la parte espacial factorizada solo escala ×4. Comprueba: $64 \cdot 1024^2 + 1024 \cdot 64^2 = 67\,108\,864 + 4\,194\,304 \approx 7.1 \times 10^7$ frente a $(64 \cdot 1024)^2 \approx 4.3 \times 10^9$ — razón ≈ 60×.

📊 Propiedades y comparación

Enfoque	Consistencia temporal	Costo	Datos que exige	Limitación característica
Frames independientes (t2i por frame)	ninguna (parpadeo)	bajo	solo imagen	inutilizable como video
Difusión de video con atención completa 3D	máxima	$O((F \cdot h \cdot w)^2)$	video masivo	prohibitivo en clips largos
Difusión latente + atención factorizada	alta	$F \cdot (hw)^2 + hw \cdot F^2$	imagen + video	interacciones espacio-temporales indirectas
Video2video / propagación de ediciones	hereda del video fuente	medio	un video fuente	oclusiones y cambios de plano

⚠️ Errores conceptuales frecuentes

1. **"Generar video es generar muchas imágenes."** Sin acoplamiento temporal cada frame es plausible pero el conjunto parpadea: la identidad y la física se rompen entre frames. La temporalidad es una restricción adicional, no un bucle for.
2. **"La atención factorizada es equivalente a la completa, solo más barata."** No: restringe qué tokens se ven directamente. Es una aproximación que funciona bien empíricamente, pero movimientos que acoplan posición y tiempo de forma compleja pueden degradarse.
3. **"Más frames por segundo = mejor modelo."** La tasa de frames se puede interpolar a posteriori; lo difícil es la coherencia de largo plazo (segundos), no la densidad temporal local.
4. **"Editar un video es editar su primer frame."** La propagación exige correspondencias válidas entre frames; oclusiones, apariciones de objetos nuevos y cortes de plano rompen la propagación y exigen re-anclar la edición.
5. **"Un buen FVD garantiza un buen video."** FVD compara distribuciones de features, no verifica física ni causalidad; un modelo puede lograr FVD bajo y aun así generar manos imposibles o relojes que giran al revés.

🚀 Del aprendizaje a la operación

Entre este núcleo educativo y un producto real de video generativo faltan: un pipeline de curación de datos con licencias y filtrado de contenido, infraestructura de inferencia con presupuesto de GPU realista (minutos de cómputo por segundos de video), control de identidad persistente entre tomas (personajes consistentes), integración de marcas de procedencia (C2PA) en el contenedor de video exportado, y revisión humana del material generado antes de cualquier publicación.

🧪 Laboratorio

```
python lab.py
```

El laboratorio llama a `ai_evolution.labs.run_lab("generation")`. Esta decisión evita una implementación divergente por clase: cada una conserva un entrypoint propio, pero los motores didácticos se prueban como una biblioteca común.

🔍 Evidencia esperada

- tipo de laboratorio y semilla;
- entradas o decisiones observables;
- resultado estructurado;
- lista `evidence` con hechos que pueden inspeccionarse;
- lista `limitations` que impide presentar la demo como producción.

📓 Notebooks

- 📓 `notebook.ipynb`: recorrido guiado con la materia resumida.
- ✍️ `notebook_student.ipynb`: ejercicios para resolver.

- ✅ `notebook_solution.ipynb`: solución de referencia explicada.

📝 Evaluación

Criterio	Peso
Comprensión conceptual	25 %
Ejecución reproducible	25 %
Interpretación basada en evidencia	25 %
Riesgos, límites y mejora propuesta	25 %

Consulta assessment.md para preguntas y criterio de aceptación.

⚠️ Errores comunes

Síntoma	Causa probable	Corrección
El código corre, pero no hay conclusión	Se confundió ejecución con aprendizaje	Explica qué demuestra y qué no demuestra
El resultado cambia sin explicación	No se registró semilla o configuración	Conserva semilla, versión y parámetros
Se promete uso real	Se extrapoló desde una demo educativa	Declara entorno, datos, límites y revisión humana
Se copia una métrica aislada	No existe baseline ni costo de error	Añade comparación y criterio de decisión

❓ Preguntas frecuentes

¿Debo usar una API comercial?
No. El núcleo funciona localmente. Las extensiones LIVE se documentan por separado.

¿El laboratorio representa una implementación industrial?
No por sí solo. Enseña el contrato y el patrón; producción exige integración, seguridad, observabilidad, pruebas y operación.

¿Dónde profundizo?
Revisa las especializaciones enlazadas en el README raíz y la ruta siguiente.

🔗 Referencias

- Ho, J. et al. (2022). *Video Diffusion Models*. arXiv:2204.03458 — uso: fuente primaria del mecanismo estudiado
- Singer, U. et al. (2022). *Make-A-Video: Text-to-Video Generation without Text-Video Data*. arXiv:2209.14792 — uso: fuente primaria del mecanismo estudiado
- Blattmann, A. et al. (2023). *Stable Video Diffusion: Scaling Latent Video Diffusion Models to Large Datasets*. arXiv:2311.15127 — uso: fuente primaria del mecanismo estudiado

- Rombach, R. et al. (2022). *High-Resolution Image Synthesis with Latent Diffusion Models*. arXiv:2112.10752 — uso: fuente primaria del mecanismo estudiado
- Ho, J., Jain, A. y Abbeel, P. (2020). *Denoising Diffusion Probabilistic Models*. arXiv:2006.11239 — uso: fuente primaria del mecanismo estudiado

---

📜 Papers que fundamentan esta clase

> Bloque generado por `python scripts/link_papers_to_classes.py`. La fuente es `papers/catalog/papers.json`.

Paper	Año	Qué desbloqueó	Miniatura
P17 · Modelos probabilísticos de difusión con eliminación de ruido	2020	La generación deja de ser un salto en la oscuridad: se aprende a deshacer, paso a paso, un proceso de ruido conocido.	notebook

Cada ficha explica el problema anterior, la matemática mínima, los límites y los errores de atribución más frecuentes. Para leerlas con método: cómo leer un paper de IA · anexos matemáticos.

📚 Bibliografía de apoyo

> Bloque generado por `python scripts/link_sources_to_classes.py`. Cada obra lleva su localizador verificado en `sources/bibliography.json`.

Los papers dicen **de dónde salió** el mecanismo. Estas obras lo **desarrollan** con el espacio que una clase no tiene: teoría completa, demostraciones y ejercicios.

Obra	Edición	Localizador	Papel en esta clase
Goodfellow, Ian, Bengio, Yoshua y Courville, Aaron — *Deep Learning*	2016	ISBN 9780262035613 · web de la obra	obra de referencia de la parte 07 · capítulos de modelos generativos

📝 Evaluación completa

❓ Preguntas

1. Define generación y edición de video sin usar una marca o framework como definición.
2. Explica la relación entre video, temporalidad, consistencia, edición.
3. Ejecuta `lab.py` dos veces con la misma semilla. ¿Qué debe conservarse?
4. Identifica una afirmación permitida y una afirmación exagerada sobre el resultado.
5. Propón una prueba negativa o un caso límite.

🏆 Reto verificable

Amplía el resultado del laboratorio con una clave `student_extension` que incluya:

- el supuesto que estás probando;
- una medición o comprobación;
- la conclusión;
- una limitación.

✅ Criterio de aceptación

- [ ] `lab.py` termina con código 0.
- [ ] El resultado contiene `kind`, `seed`, `evidence` y `limitations`.
- [ ] La extensión no modifica el comportamiento de otras clases.
- [ ] La interpretación referencia datos impresos por el laboratorio.
- [ ] Se declara al menos un riesgo o condición de no uso.

---

096 — Generación 3D y mundos sintéticos

Parte: 07 — IA generativa para texto, imagen, audio, video y 3D
Nivel: avanzado · **Horas estimadas:** 6
Laboratorio: `generation` · **Estado:** `EXECUTABLE_CORE`

🎯 Propósito

Comprender **generación 3d y mundos sintéticos** dentro de la evolución de la inteligencia artificial, implementar un experimento mínimo verificable y distinguir qué parte constituye evidencia frente a una afirmación todavía no comprobada.

📚 Resultados de aprendizaje

Al finalizar podrás:

1. Explicar generación 3d y mundos sintéticos usando los conceptos `3D`, `NeRF`, `gaussian splatting`, `assets`.
2. Ejecutar el laboratorio con una semilla explícita y revisar su contrato JSON.
3. Identificar al menos un supuesto, una limitación y un riesgo de aplicación.
4. Comparar el enfoque con la etapa anterior de la ruta de aprendizaje.
5. Producir una evidencia reproducible y una conclusión que no exceda los datos.

🧩 Conceptos centrales

`3D`, `NeRF`, `gaussian splatting`, `assets`

🗺️ Ubicación en el mapa de la IA

La generación 3D cierra el arco de esta parte: tras texto, imagen, audio y video, el objetivo es sintetizar **escenas** con geometría consistente desde cualquier punto de vista. NeRF (2020) reformuló la reconstrucción 3D como aprender un campo continuo con una red neuronal; 3D Gaussian Splatting (2023) recuperó las primitivas explícitas para lograr render en tiempo real; y DreamFusion (2022) conectó ambos mundos con los modelos de difusión de imagen (clase 090) para generar 3D a partir de texto **sin datos 3D**. Estas técnicas alimentan mundos sintéticos para simulación, robótica y datos de entrenamiento (clase 097).

📖 Fundamentos

🌫️ NeRF: la escena como campo de radiancia

Un *Neural Radiance Field* representa la escena como una función continua aprendida por un MLP:

```
F(x, d) → (c, σ)
  x = (x, y, z)   posición en el espacio
  d = (θ, ϕ)      dirección de vista
  c = (r, g, b)   color emitido en x visto desde d
  σ ≥ 0           densidad volumétrica en x (opacidad diferencial)
```

La densidad σ depende solo de la posición (la geometría no cambia con el observador); el color depende también de la dirección, lo que permite reflejos y brillos especulares. Las coordenadas se pasan por un *positional encoding* (senos y cosenos a frecuencias crecientes) porque un MLP crudo no representa bien detalles de alta frecuencia.

🎥 Render volumétrico: de campo a píxel

Para pintar un píxel se lanza un rayo $r(t) = o + t \cdot d$ desde la cámara y se integra el campo a lo largo del rayo. En forma discreta, con N muestras a distancias δ_i entre muestras consecutivas:

```
αᵢ = 1 − exp(−σᵢ·δᵢ)                  opacidad del segmento i
Tᵢ = exp(−Σ_{j<i} σⱼ·δⱼ)              transmitancia: luz que llega hasta i
wᵢ = Tᵢ · αᵢ                          peso de la muestra i
C  = Σᵢ wᵢ · cᵢ                        color final del píxel
```

La transmitancia T_i decae a medida que el rayo atraviesa materia: una muestra detrás de una superficie densa contribuye poco aunque su σ sea alto. Todo el proceso es diferenciable, así que se entrena minimizando el error fotométrico entre el píxel renderizado y la fotografía real, con decenas de imágenes calibradas de la escena. Costo característico: cientos de evaluaciones del MLP **por píxel** — entrenamiento en horas y render lejos del tiempo real en el NeRF original.

💎 3D Gaussian Splatting: primitivas explícitas y rasterización

3DGS (Kerbl et al., 2023) sustituye el campo implícito por millones de **gaussianas 3D explícitas**, cada una con: posición μ, covarianza Σ (forma y orientación del elipsoide), opacidad α y color dependiente de la vista (armónicos esféricos). El render no integra a lo largo de rayos: **proyecta** cada gaussiana al plano de imagen (*splatting*), la ordena por profundidad y compone con alpha-blending — el mismo principio de composición que NeRF, pero rasterizado en GPU. Resultado: entrenamiento en minutos y render a >100 fps con calidad comparable. El precio: una nube de millones de primitivas (memoria), y una densificación/poda heurística durante el entrenamiento que hay que ajustar.

✨ Text-to-3D: DreamFusion y Score Distillation Sampling

DreamFusion (Poole et al., 2022) genera un objeto 3D a partir de texto **sin ningún dataset 3D**. La idea, *Score Distillation Sampling* (SDS):

```
repetir:
  1. renderizar el NeRF desde una cámara aleatoria → imagen I
  2. añadir ruido a I y pedir a un modelo de difusión texto-imagen congelado
     que estime ese ruido, condicionado en el prompt
  3. usar (ruido_estimado − ruido_real) como gradiente sobre los parámetros del NeRF
```

El modelo de difusión actúa como crítico: empuja cada vista renderizada hacia la distribución de imágenes que corresponde al prompt. Como todas las vistas comparten la misma escena 3D, la consistencia geométrica emerge. Limitaciones conocidas: saturación de color y sobre-suavizado (el guidance alto promedia modos), y el problema *Janus* — caras repetidas en varios lados del objeto, porque el crítico 2D no sabe desde dónde está mirando.

🧮 Ejemplo trabajado

Render volumétrico a mano con **3 muestras** a lo largo de un rayo, todas con $\delta_i = 1.0$, densidades $\sigma = (0.5, 1.0, 2.0)$ y colores RGB $c_1 = (1, 0, 0)$, $c_2 = (0, 1, 0)$, $c_3 = (0, 0, 1)$:

```
Opacidades:      α₁ = 1 − e^(−0.5) = 0.3935
                 α₂ = 1 − e^(−1.0) = 0.6321
                 α₃ = 1 − e^(−2.0) = 0.8647

Transmitancias:  T₁ = e^0        = 1.0000   (nada delante)
                 T₂ = e^(−0.5)   = 0.6065
                 T₃ = e^(−1.5)   = 0.2231   (0.5 + 1.0 acumulado)

Pesos:           w₁ = 1.0000 · 0.3935 = 0.3935
                 w₂ = 0.6065 · 0.6321 = 0.3834
                 w₃ = 0.2231 · 0.8647 = 0.1929
                 Σw = 0.9698  →  1 − Σw = 0.0302 llega al fondo

Color final:     C = 0.3935·(1,0,0) + 0.3834·(0,1,0) + 0.1929·(0,0,1)
                   = (0.394, 0.383, 0.193)
```

Lectura: aunque la tercera muestra es la más densa ($\sigma = 2$), pesa **menos** que las dos primeras porque la materia que tiene delante ya absorbió el 78 % de la luz del rayo. Si duplicas σ_1 a 1.0, verás que T_2 y T_3 caen y el rojo domina aún más: la oclusión es multiplicativa, no aditiva.

📊 Propiedades y comparación

Método	Representación	Entrenamiento	Render	Memoria	Limitación típica
NeRF	implícita (MLP)	horas	segundos/frame	MB (pesos)	lento; no editable directamente
3D Gaussian Splatting	explícita (gaussianas)	minutos	>100 fps	cientos de MB-GB	heurísticas de densificación; escenas enormes
DreamFusion (SDS)	NeRF guiado por difusión 2D	horas por objeto	el del NeRF	MB	Janus, sobre-saturación; sin datos 3D reales
Fotogrametría clásica (SfM+MVS)	malla + textura	horas	tiempo real	según malla	falla en superficies brillantes/sin textura

⚠️ Errores conceptuales frecuentes

1. **"NeRF almacena una malla o vóxeles."** No almacena geometría explícita: la escena *es* los pesos del MLP. La superficie solo existe implícitamente donde σ es alta, y extraer una malla requiere un paso adicional (p. ej. marching cubes).
2. **"Más densidad σ implica más contribución al píxel."** Solo si la muestra está delante: el peso es $T_i \cdot \alpha_i$, y la transmitancia castiga todo lo que queda detrás de materia densa — como muestra el ejemplo trabajado.
3. **"Gaussian Splatting es un NeRF más rápido."** Cambia la representación (explícita vs implícita) y el algoritmo de render (rasterización vs integración por rayo); comparten la composición alpha, no la arquitectura.
4. **"DreamFusion aprende de un dataset 3D."** Precisamente no: destila un modelo de difusión 2D. Esa es su fortaleza (no necesita 3D) y su debilidad (el crítico 2D ignora la vista, de ahí el problema Janus).
5. **"Si cada vista renderizada se ve bien, la geometría es correcta."** Vistas plausibles pueden esconder geometría degenerada ("floaters", superficies huecas); la calidad 3D se evalúa con vistas *no vistas* en entrenamiento y con la geometría extraída, no con las vistas de ajuste.

🚀 Del aprendizaje a la operación

Entre este núcleo educativo y un pipeline 3D real faltan: captura calibrada (poses de cámara vía SfM, que falla en escenas sin textura), conversión de la representación a assets utilizables (mallas con topología limpia, UVs y LODs para un motor de juego), presupuestos de memoria y streaming para escenas grandes, control de derechos sobre las escenas y objetos capturados (propiedad y privacidad de espacios reales), y validación geométrica independiente cuando el 3D alimenta simulación o robótica.

🧪 Laboratorio

```
python lab.py
```

El laboratorio llama a `ai_evolution.labs.run_lab("generation")`. Esta decisión evita una implementación divergente por clase: cada una conserva un entrypoint propio, pero los motores didácticos se prueban como una biblioteca común.

🔍 Evidencia esperada

- tipo de laboratorio y semilla;
- entradas o decisiones observables;
- resultado estructurado;
- lista `evidence` con hechos que pueden inspeccionarse;
- lista `limitations` que impide presentar la demo como producción.

📓 Notebooks

- 📓 `notebook.ipynb`: recorrido guiado con la materia resumida.
- ✍️ `notebook_student.ipynb`: ejercicios para resolver.
- ✅ `notebook_solution.ipynb`: solución de referencia explicada.

📝 Evaluación

Criterio	Peso
Comprensión conceptual	25 %
Ejecución reproducible	25 %
Interpretación basada en evidencia	25 %
Riesgos, límites y mejora propuesta	25 %

Consulta assessment.md para preguntas y criterio de aceptación.

⚠️ Errores comunes

Síntoma	Causa probable	Corrección
El código corre, pero no hay conclusión	Se confundió ejecución con aprendizaje	Explica qué demuestra y qué no demuestra
El resultado cambia sin explicación	No se registró semilla o configuración	Conserva semilla, versión y parámetros
Se promete uso real	Se extrapoló desde una demo educativa	Declara entorno, datos, límites y revisión humana
Se copia una métrica aislada	No existe baseline ni costo de error	Añade comparación y criterio de decisión

❓ Preguntas frecuentes

¿Debo usar una API comercial?
No. El núcleo funciona localmente. Las extensiones LIVE se documentan por separado.

¿El laboratorio representa una implementación industrial?
No por sí solo. Enseña el contrato y el patrón; producción exige integración, seguridad, observabilidad, pruebas y operación.

¿Dónde profundizo?
Revisa las especializaciones enlazadas en el README raíz y la ruta siguiente.

🔗 Referencias

- Mildenhall, B. et al. (2020). *NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis*. arXiv:2003.08934 — uso: fuente primaria del mecanismo estudiado
- Kerbl, B., Kopanas, G., Leimkühler, T. y Drettakis, G. (2023). *3D Gaussian Splatting for Real-Time Radiance Field Rendering*. arXiv:2308.04079 — uso: fuente primaria del mecanismo estudiado
- Poole, B., Jain, A., Barron, J. T. y Mildenhall, B. (2022). *DreamFusion: Text-to-3D using 2D Diffusion*. arXiv:2209.14988 — uso: fuente primaria del mecanismo estudiado
- Müller, T. et al. (2022). *Instant Neural Graphics Primitives with a Multiresolution Hash Encoding* (Instant-NGP). arXiv:2201.05989 — uso: fuente primaria del mecanismo estudiado
- Rombach, R. et al. (2022). *High-Resolution Image Synthesis with Latent Diffusion Models*. arXiv:2112.10752 — uso: fuente primaria del mecanismo estudiado

---

📜 Papers que fundamentan esta clase

> Bloque generado por `python scripts/link_papers_to_classes.py`. La fuente es `papers/catalog/papers.json`.

Paper	Año	Qué desbloqueó	Miniatura
P128 · NeRF: representar escenas como campos de radiancia neuronal	2020	Sustituye la escena explícita por una función continua que un perceptrón representa, y sintetiza vistas nuevas con una fidelidad que no se había visto.	notebook
P132 · Splatting de gaussianas 3D para renderizado de campos de radiancia en tiempo real	2023	Alcanza calidad de campo de radiancia a velocidad de tiempo real cambiando la función continua por millones de primitivas explícitas que se rasterizan.	notebook

Cada ficha explica el problema anterior, la matemática mínima, los límites y los errores de atribución más frecuentes. Para leerlas con método: cómo leer un paper de IA · anexos matemáticos.

📚 Bibliografía de apoyo

> Bloque generado por `python scripts/link_sources_to_classes.py`. Cada obra lleva su localizador verificado en `sources/bibliography.json`.

Los papers dicen **de dónde salió** el mecanismo. Estas obras lo **desarrollan** con el espacio que una clase no tiene: teoría completa, demostraciones y ejercicios.

Obra	Edición	Localizador	Papel en esta clase
Goodfellow, Ian, Bengio, Yoshua y Courville, Aaron — *Deep Learning*	2016	ISBN 9780262035613 · web de la obra	obra de referencia de la parte 07 · capítulos de modelos generativos

📝 Evaluación completa

❓ Preguntas

1. Define generación 3d y mundos sintéticos sin usar una marca o framework como definición.
2. Explica la relación entre 3D, NeRF, gaussian splatting, assets.
3. Ejecuta `lab.py` dos veces con la misma semilla. ¿Qué debe conservarse?
4. Identifica una afirmación permitida y una afirmación exagerada sobre el resultado.
5. Propón una prueba negativa o un caso límite.

🏆 Reto verificable

Amplía el resultado del laboratorio con una clave `student_extension` que incluya:

- el supuesto que estás probando;
- una medición o comprobación;
- la conclusión;
- una limitación.

✅ Criterio de aceptación

- [ ] `lab.py` termina con código 0.
- [ ] El resultado contiene `kind`, `seed`, `evidence` y `limitations`.
- [ ] La extensión no modifica el comportamiento de otras clases.
- [ ] La interpretación referencia datos impresos por el laboratorio.
- [ ] Se declara al menos un riesgo o condición de no uso.

---

097 — Datos sintéticos: utilidad y contaminación

Parte: 07 — IA generativa para texto, imagen, audio, video y 3D
Nivel: avanzado · **Horas estimadas:** 6
Laboratorio: `evaluation` · **Estado:** `EXECUTABLE_CORE`

🎯 Propósito

Comprender **datos sintéticos: utilidad y contaminación** dentro de la evolución de la inteligencia artificial, implementar un experimento mínimo verificable y distinguir qué parte constituye evidencia frente a una afirmación todavía no comprobada.

📚 Resultados de aprendizaje

Al finalizar podrás:

1. Explicar datos sintéticos: utilidad y contaminación usando los conceptos `synthetic data`, `leakage`, `collapse`, `calidad`.
2. Ejecutar el laboratorio con una semilla explícita y revisar su contrato JSON.
3. Identificar al menos un supuesto, una limitación y un riesgo de aplicación.
4. Comparar el enfoque con la etapa anterior de la ruta de aprendizaje.
5. Producir una evidencia reproducible y una conclusión que no exceda los datos.

🧩 Conceptos centrales

`synthetic data`, `leakage`, `collapse`, `calidad`

🗺️ Ubicación en el mapa de la IA

Las clases 088–096 mostraron cómo generar texto, imagen, audio, video y 3D. Esta clase cierra el círculo: esas salidas generadas vuelven a entrar como *datos de entrenamiento*. Los datos sintéticos son hoy un insumo estándar (aumento, privacidad, balanceo), pero desde 2023–2024 sabemos que el ciclo tiene un modo de fallo propio —el *model collapse* demostrado por Shumailov et al. en *Nature*— que convierte la procedencia de los datos (clase 098) en un problema de ingeniería, no solo de ética.

📖 Fundamentos

🏭 Qué son los datos sintéticos y para qué se usan

Datos sintéticos son ejemplos producidos por un modelo generativo o un procedimiento estadístico, no recolectados del fenómeno real. Tres usos legítimos y bien delimitados:

1. **Aumento de datos (augmentation):** ampliar un conjunto pequeño con variantes plausibles (rotaciones, paráfrasis, muestras de un generador) para reducir sobreajuste.

2. **Privacidad:** publicar un sustituto sintético de un dataset sensible (salud, finanzas) que preserve estadísticas agregadas sin exponer registros individuales. Ojo: sin garantías formales (p. ej. privacidad diferencial), "sintético" no implica "anónimo".
3. **Balanceo de clases:** generar ejemplos de la clase minoritaria. El método clásico es **SMOTE** (Chawla et al., 2002): interpola entre una instancia minoritaria y uno de sus k vecinos más cercanos:

```
x_nuevo = x_i + λ · (x_vecino − x_i),   λ ~ Uniforme(0, 1)
```

📏 Medir la utilidad: TSTR

La fidelidad visual o estadística no basta: la pregunta operativa es si los datos sintéticos *sirven para entrenar*. El protocolo estándar es **TSTR** (*Train on Synthetic, Test on Real*):

```
1. Entrena el modelo M_s con datos sintéticos.
2. Entrena el modelo M_r con datos reales (baseline).
3. Evalúa AMBOS sobre un conjunto de prueba REAL nunca visto.
4. Utilidad ≈ métrica(M_s) / métrica(M_r)   (idealmente cercana a 1)
```

Un TSTR alto exige que el generador capture la relación entre features y etiqueta, no solo las distribuciones marginales. El protocolo inverso (TRTS: entrenar con real, evaluar con sintético) mide otra cosa —si lo sintético es *reconocible* por un modelo real— y no debe confundirse con utilidad.

☣️ Contaminación de corpus y model collapse

Contaminación (leakage generativo): el contenido generado se publica en la web, se recolecta en el siguiente crawl y termina en el corpus de entrenamiento de la próxima generación de modelos, sin marca que lo distinga. **Model collapse** (Shumailov et al., *Nature* 2024) es la degeneración que aparece al entrenar generaciones sucesivas sobre salidas de la generación anterior. El mecanismo combina tres errores que se acumulan:

- **Error de aproximación estadística:** cada generación aprende de una *muestra finita* de la anterior; los eventos raros (colas de la distribución) pueden no aparecer en la muestra y, si no aparecen, el nuevo modelo les asigna probabilidad ≈ 0.
- **Error de expresividad:** el modelo no puede representar exactamente la distribución objetivo, y el sesgo se hereda.
- **Error de aprendizaje:** el procedimiento de optimización añade su propio sesgo (p. ej. hacia salidas de alta probabilidad).

El resultado tiene dos fases: **colapso temprano** (se pierden las colas: lo raro desaparece) y **colapso tardío** (la distribución converge a una versión estrecha y de baja varianza, poco parecida a la original). La propiedad clave es que la pérdida de una cola es un **estado absorbente**: si en una generación la clase rara no se muestrea, ninguna generación posterior puede recuperarla a partir de esos datos.

🧮 Ejemplo trabajado

Simulemos a mano el colapso con una distribución discreta de dos clases: `común` con p = 0.95 y `rara` con p = 0.05. Cada generación muestrea **N = 20** ejemplos de la generación anterior y reestima las probabilidades por

frecuencia.

Generación 0 → 1. Con p_rara = 0.05 y N = 20, el número esperado de ejemplos raros es 20 · 0.05 = 1. Pero la probabilidad de obtener **cero** ejemplos raros es:

```
P(0 raros) = (1 − 0.05)²⁰ = 0.95²⁰ ≈ 0.358
```

Es decir: en ~36 % de los mundos posibles, la clase rara desaparece en una sola generación y la nueva estimación es $\hat{p}$_rara = 0/20 = 0.

Generación 1 → 2. Si sobrevivió con 1/20 ($\hat{p}$ = 0.05), el riesgo se repite: otra vez ≈ 35.8 % de extinción. Si murió, $\hat{p}$ = 0 es absorbente: se muestrea solo `común`.

Generación 2 → 3. Probabilidad de que la cola siga viva tras 3 generaciones (aproximando $\hat{p}$ ≈ 0.05 mientras sobreviva):

```
P(sobrevive 3 gen) ≈ (1 − 0.358)³ ≈ 0.642³ ≈ 0.265
```

Con solo tres generaciones recursivas, en ~3 de cada 4 corridas la clase rara ya no existe y el "modelo" final asigna p = 0 a algo que en la realidad ocurre el 5 % de las veces. Con distribuciones continuas el efecto es análogo: la varianza estimada se contrae generación tras generación.

📊 Propiedades y comparación

Fuente de datos	Utilidad (TSTR)	Riesgo de privacidad	Riesgo de colapso	Costo
Reales curados	referencia (≈ 1)	alto si son sensibles	ninguno	alto (recolección, licencias)
SMOTE / interpolación	media (solo clases minoritarias)	medio (interpola registros reales)	bajo (una sola pasada)	muy bajo
Generador profundo (1 generación)	media-alta si el generador es bueno	medio (memorización posible)	bajo	medio
Recursivo (gen n sobre gen n−1)	decreciente con n	bajo	**alto: pérdida de colas**	bajo (y por eso tentador)

⚠️ Errores conceptuales frecuentes

1. **"Si los datos sintéticos se ven realistas, sirven para entrenar."** Fidelidad perceptual ≠ utilidad. TSTR puede ser pobre aunque las muestras engañen al ojo, porque la relación feature–etiqueta no se preservó.
2. **"Sintético = anónimo."** Los generadores memorizan: un modelo entrenado con datos sensibles puede reproducir registros casi idénticos. Sin garantías formales (privacidad diferencial), no hay anonimato.
3. **"El colapso es un problema de modelos malos."** Es un problema de *muestreo finito*: incluso un estimador perfecto pierde las colas con probabilidad positiva en cada generación, y la pérdida es irreversible dentro del ciclo.
4. **"Mezclar un poco de dato real lo arregla todo."** Conservar una fracción de datos reales frescos mitiga el colapso (lo muestra el propio paper de Shumailov et al.), pero exige saber *cuáles* datos son reales: sin procedencia (clase 098) no puedes aplicar la mitigación.
5. **"SMOTE genera información nueva."** SMOTE interpola entre puntos existentes: no añade información sobre regiones no observadas y puede crear ejemplos irreales si la clase minoritaria no es convexa.

🚀 Del aprendizaje a la operación

Entre esta simulación y un pipeline real de datos sintéticos faltan: un protocolo TSTR con validación cruzada y varios modelos downstream, auditoría de memorización (búsqueda de vecinos casi idénticos entre sintético y real), etiquetado de procedencia de cada lote (¿qué fracción del corpus es generada y por qué modelo?), y una política explícita de mezcla real/sintético con presupuesto de datos frescos por generación. La detección de contaminación en corpus web abiertos sigue siendo un problema abierto.

🧪 Laboratorio

```
python lab.py
```

El laboratorio llama a `ai_evolution.labs.run_lab("evaluation")`. Esta decisión evita una implementación divergente por clase: cada una conserva un entrypoint propio, pero los motores didácticos se prueban como una biblioteca común.

🔍 Evidencia esperada

- tipo de laboratorio y semilla;
- entradas o decisiones observables;
- resultado estructurado;
- lista `evidence` con hechos que pueden inspeccionarse;
- lista `limitations` que impide presentar la demo como producción.

📓 Notebooks

- 📓 `notebook.ipynb`: recorrido guiado con la materia resumida.
- ✍️ `notebook_student.ipynb`: ejercicios para resolver.
- ✅ `notebook_solution.ipynb`: solución de referencia explicada.

📝 Evaluación

Criterio	Peso
Comprensión conceptual	25 %
Ejecución reproducible	25 %
Interpretación basada en evidencia	25 %
Riesgos, límites y mejora propuesta	25 %

Consulta assessment.md para preguntas y criterio de aceptación.

⚠️ Errores comunes

Síntoma	Causa probable	Corrección
El código corre, pero no hay conclusión	Se confundió ejecución con aprendizaje	Explica qué demuestra y qué no demuestra
El resultado cambia sin explicación	No se registró semilla o configuración	Conserva semilla, versión y parámetros
Se promete uso real	Se extrapoló desde una demo educativa	Declara entorno, datos, límites y revisión humana
Se copia una métrica aislada	No existe baseline ni costo de error	Añade comparación y criterio de decisión

❓ Preguntas frecuentes

¿Debo usar una API comercial?
No. El núcleo funciona localmente. Las extensiones LIVE se documentan por separado.

¿El laboratorio representa una implementación industrial?
No por sí solo. Enseña el contrato y el patrón; producción exige integración, seguridad, observabilidad, pruebas y operación.

¿Dónde profundizo?
Revisa las especializaciones enlazadas en el README raíz y la ruta siguiente.

🔗 Referencias

- Shumailov, I. et al. (2024). *AI models collapse when trained on recursively generated data*. Nature, 631. doi:10.1038/s41586-024-07566-y · versión arXiv — uso: fuente primaria del mecanismo estudiado
- Chawla, N. et al. (2002). *SMOTE: Synthetic Minority Over-sampling Technique*. JAIR, 16. doi:10.1613/jair.953 — uso: fuente primaria del mecanismo estudiado
- Goodfellow, I., Bengio, Y. y Courville, A. (2016). *Deep Learning*, cap. 20 (Deep Generative Models). deeplearningbook.org/contents/generative_models.html — uso: desarrollo extendido del tema
- Goodfellow, I. et al. (2014). *Generative Adversarial Networks*. arXiv:1406.2661 — uso: fuente primaria del mecanismo estudiado

- C2PA (procedencia como mitigación de contaminación, ver clase 098): especificación 2.2 — uso: marco normativo de referencia

---

📜 Papers que fundamentan esta clase

> Bloque generado por `python scripts/link_papers_to_classes.py`. La fuente es `papers/catalog/papers.json`.

Paper	Año	Qué desbloqueó	Miniatura
P133 · Los modelos de IA colapsan al entrenarse con datos generados recursivamente	2024	Demuestra que entrenar generación tras generación con datos sintéticos estrecha la distribución de forma irreversible, sin que ningún modelo cometa error alguno.	notebook

Cada ficha explica el problema anterior, la matemática mínima, los límites y los errores de atribución más frecuentes. Para leerlas con método: cómo leer un paper de IA · anexos matemáticos.

📚 Bibliografía de apoyo

> Bloque generado por `python scripts/link_sources_to_classes.py`. Cada obra lleva su localizador verificado en `sources/bibliography.json`.

Los papers dicen **de dónde salió** el mecanismo. Estas obras lo **desarrollan** con el espacio que una clase no tiene: teoría completa, demostraciones y ejercicios.

Obra	Edición	Localizador	Papel en esta clase
Goodfellow, Ian, Bengio, Yoshua y Courville, Aaron — *Deep Learning*	2016	ISBN 9780262035613 · web de la obra	citada en las referencias de esta clase · cap. 20 · obra de referencia de la parte 07

Normas y documentación oficial que aplica esta clase: c2pa.org/specifications

📝 Evaluación completa

❓ Preguntas

1. Define datos sintéticos: utilidad y contaminación sin usar una marca o framework como definición.
2. Explica la relación entre synthetic data, leakage, collapse, calidad.
3. Ejecuta `lab.py` dos veces con la misma semilla. ¿Qué debe conservarse?
4. Identifica una afirmación permitida y una afirmación exagerada sobre el resultado.
5. Propón una prueba negativa o un caso límite.

🏆 Reto verificable

Amplía el resultado del laboratorio con una clave `student_extension` que incluya:

- el supuesto que estás probando;
- una medición o comprobación;
- la conclusión;
- una limitación.

✅ Criterio de aceptación

- [ ] `lab.py` termina con código 0.
- [ ] El resultado contiene `kind`, `seed`, `evidence` y `limitations`.
- [ ] La extensión no modifica el comportamiento de otras clases.
- [ ] La interpretación referencia datos impresos por el laboratorio.
- [ ] Se declara al menos un riesgo o condición de no uso.

---

098 — Procedencia, marcas y autenticidad

Parte: 07 — IA generativa para texto, imagen, audio, video y 3D
Nivel: avanzado · **Horas estimadas:** 6
Laboratorio: `safety` · **Estado:** `EXECUTABLE_CORE`

🎯 Propósito

Comprender **procedencia, marcas y autenticidad** dentro de la evolución de la inteligencia artificial, implementar un experimento mínimo verificable y distinguir qué parte constituye evidencia frente a una afirmación todavía no comprobada.

📚 Resultados de aprendizaje

Al finalizar podrás:

1. Explicar procedencia, marcas y autenticidad usando los conceptos `C2PA`, `watermark`, `provenance`, `autenticidad`.
2. Ejecutar el laboratorio con una semilla explícita y revisar su contrato JSON.
3. Identificar al menos un supuesto, una limitación y un riesgo de aplicación.
4. Comparar el enfoque con la etapa anterior de la ruta de aprendizaje.
5. Producir una evidencia reproducible y una conclusión que no exceda los datos.

🧩 Conceptos centrales

`C2PA`, `watermark`, `provenance`, `autenticidad`

🗺️ Ubicación en el mapa de la IA

Cuando los generadores de las clases 088–096 producen contenido indistinguible del humano, la pregunta "¿quién hizo esto y cómo?" deja de poder responderse mirando el contenido. Esta clase estudia las dos respuestas de la ingeniería: **procedencia activa** (C2PA: metadatos firmados criptográficamente que viajan con el activo) y **marcas de agua estadísticas** (SynthID, esquemas tipo Kirchenbauer: sesgos imperceptibles insertados durante la generación). Ambas alimentan directamente el pipeline trazable del proyecto integrador (clase 099) y la mitigación de la contaminación de corpus (clase 097).

📖 Fundamentos

📜 Procedencia activa: C2PA

C2PA (Coalition for Content Provenance and Authenticity) define un formato de **manifiesto** que se adjunta al activo (imagen, video, audio, PDF) y registra su historia. La estructura esencial:

- **Aserciones (assertions):** hechos sobre el activo — qué herramienta lo creó, qué modelo generativo intervino, qué ediciones se aplicaron, miniaturas, EXIF.

- **Reclamo (claim):** agrupa las aserciones y contiene el **hard binding**: un hash criptográfico del contenido del activo que liga el manifiesto a *esos bytes exactos*.
- **Firma del reclamo (claim signature):** firma digital del reclamo con el certificado X.509 del firmante (fabricante de cámara, editor de software, servicio de IA).

Cada edición añade un nuevo manifiesto que referencia (y contiene el hash de) el manifiesto anterior, formando una **cadena de procedencia**. Verificar = recalcular el hash del activo, compararlo con el hard binding, y validar las firmas de toda la cadena. Cualquier alteración de los bytes rompe el hash; cualquier alteración del manifiesto rompe la firma.

```
verificar(activo):
    m ← extraer_manifiesto(activo)
    si hash(contenido) ≠ m.hard_binding → INVÁLIDO (contenido alterado)
    para cada eslabón de la cadena:
        si no verifica_firma(eslabón) → INVÁLIDO (manifiesto alterado)
        si hash(eslabón_previo) ≠ referencia → INVÁLIDO (cadena rota)
    devolver cadena de procedencia verificada
```

Importante: C2PA acredita **quién firmó y qué proceso declaró**, no que el contenido sea "verdadero". Y su ausencia no prueba nada: la mayoría del contenido legítimo no lleva manifiesto.

💧 Marcas de agua estadísticas para texto

Un LLM genera token a token muestreando de una distribución. El esquema de **Kirchenbauer et al. (2023)** inserta una marca invisible sesgando ese muestreo:

1. Antes de emitir el token t, se usa un hash del token anterior como semilla para partir el vocabulario en una **lista verde** (fracción γ) y una **lista roja** ($1-\gamma$).
2. Se suma un sesgo $\delta > 0$ al logit de todos los tokens verdes y se muestrea normalmente.
3. El texto resultante contiene *más tokens verdes de lo esperable por azar*, sin que un lector lo note.

Detección = test de hipótesis. Bajo H_0 (texto sin marca), cada token es verde con probabilidad γ, así que el conteo de verdes k en T tokens sigue Binomial(T, γ). El estadístico:

$$z = (k - \gamma T) / \sqrt{T \cdot \gamma \cdot (1 - \gamma)}$$

Un z grande (p. ej. ≥ 4) hace H_0 insostenible. El detector solo necesita la clave del hash, no el modelo. **SynthID-Text** (Dathathri et al., *Nature* 2024) generaliza la idea con *tournament sampling* y está desplegado en producción en Gemini; SynthID también cubre imagen y audio con perturbaciones imperceptibles decodificables.

🔍 Detección pasiva vs procedencia activa

- **Detección pasiva:** clasificadores que buscan artefactos estadísticos del contenido generado *sin cooperación del generador*. Frágil: los artefactos cambian con cada modelo nuevo y las tasas de falso positivo penalizan textos atípicos (p. ej. hablantes no nativos).
- **Procedencia activa (C2PA) y marcas de agua:** requieren cooperación del generador, pero dan garantías verificables (firma) o estadísticas (z-score).

Limitaciones reales de todas las vías: el recorte y la recompresión destruyen metadatos no anclados; parafrasear o traducir un texto marcado diluye la señal verde; el **lavado de marca** (regenerar el contenido con otro modelo sin marca) la elimina; y quitar un manifiesto C2PA es trivial — la garantía es unidireccional: presencia válida ⇒ procedencia verificada; ausencia ⇏ contenido humano.

🏷️ De la procedencia al etiquetado exigible

La procedencia técnica y la divulgación al público son controles distintos. Un manifiesto C2PA o una marca legible por máquina ayuda a herramientas a reconocer origen y edición; una **etiqueta visible** ayuda a la persona a entender que interactúa con IA o que un contenido fue generado o manipulado. Ninguna de las dos demuestra que el contenido sea verdadero.

El código europeo final de 2026 ofrece pasos voluntarios para cumplir las obligaciones de transparencia del AI Act aplicables desde el 2 de agosto de 2026. Destaca tres superficies: chatbots, deepfakes y texto generado o manipulado sobre asuntos de interés público. El diseño correcto combina marca legible por máquina, aviso visible y cadena de procedencia; si una transformación elimina el metadato, debe quedar una estrategia de degradación y no una falsa afirmación de autenticidad.

🧮 Ejemplo trabajado

Test de una marca tipo greenlist con γ = 0.5 sobre un texto de **T = 100 tokens** en el que el detector cuenta **k = 70 tokens verdes**.

```
Esperado bajo H₀:      γT = 0.5 · 100 = 50 verdes
Varianza bajo H₀:      T·γ·(1−γ) = 100 · 0.5 · 0.5 = 25   →  σ = 5
Estadístico:           z = (70 − 50) / 5 = 4.0
```

Un z = 4.0 corresponde a un p-valor unilateral ≈ 3.2×10^{-5}: si el texto no tuviera marca, ver 70+ verdes ocurriría unas 3 veces en 100 000 textos. Se rechaza H_0 y se declara la marca presente. Contraste: con k = 55 verdes, z = 1.0 — perfectamente compatible con el azar; y si un parafraseador reescribe la mitad del texto llevando los verdes de 70 a ~60, z cae a 2.0 y la evidencia se vuelve débil. La detección es un *continuo de confianza*, no un sello binario.

📊 Propiedades y comparación

Mecanismo	Cooperación del generador	Garantía	Sobrevive a recompresión	Sobrevive a paráfrasis/lavado	Falsos positivos
C2PA (manifiesto firmado)	sí (firma en origen)	criptográfica	no (si se eliminan metadatos)	no (se elimina el manifiesto)	≈ 0 (firma inválida es detectable)
Marca de agua en texto (greenlist)	sí (sesgo al muestrear)	estadística (z-score)	sí (el texto es la señal)	parcial: se diluye	controlables vía umbral z
SynthID imagen/audio	sí (perturbación decodificable)	estadística	parcial (robusta a compresión moderada)	no ante regeneración	bajos, medidos
Detección pasiva	no	heurística	depende del artefacto	no	altos y sesgados

⚠️ Errores conceptuales frecuentes

1. **"C2PA demuestra que una imagen es real."** No: demuestra quién firmó qué proceso. Una imagen generada por IA puede llevar un manifiesto C2PA perfectamente válido que declara, honestamente, que fue generada.
2. **"Sin marca de agua ⇒ escrito por un humano."** La ausencia de señal no es evidencia de origen humano: la mayoría de los generadores no marcan, y las marcas se pueden lavar.
3. **"La marca de agua está en las palabras elegidas 'raras'."** El texto marcado es fluido y natural; la señal es un sesgo estadístico agregado sobre cientos de tokens, invisible token a token.
4. **"Un detector pasivo de 'texto de IA' con 95 % de accuracy es fiable."** Con prevalencia baja y costo alto del falso positivo (acusar a un estudiante), esa cifra es inutilizable; además el sesgo contra hablantes no nativos está documentado.
5. **"El hash del manifiesto protege contra el recorte."** El hard binding detecta la alteración, pero no la impide ni la revierte: un activo recortado simplemente queda *sin* procedencia verificable.

🚀 Del aprendizaje a la operación

Para operar esto de verdad faltan: una PKI con emisión y revocación de certificados de firmante (¿quién decide qué firmas son confiables?), umbrales de z calibrados sobre la tasa de falsos positivos tolerable por caso de uso, medición de robustez de la marca frente a paráfrasis y traducción con corpus adversarios, y una política de visualización para el usuario final (Content Credentials) que no sobreinterprete la ausencia de manifiesto. La estandarización (C2PA 2.x) avanza más rápido que la adopción.

🧪 Laboratorio

```
python lab.py
```

El laboratorio llama a `ai_evolution.labs.run_lab("safety")`. Esta decisión evita una implementación divergente por clase: cada una conserva un entrypoint propio, pero los motores didácticos se prueban como una biblioteca común.

🔍 Evidencia esperada

- tipo de laboratorio y semilla;
- entradas o decisiones observables;
- resultado estructurado;
- lista `evidence` con hechos que pueden inspeccionarse;
- lista `limitations` que impide presentar la demo como producción.

📓 Notebooks

- 📓 `notebook.ipynb`: recorrido guiado con la materia resumida.
- ✍️ `notebook_student.ipynb`: ejercicios para resolver.
- ✅ `notebook_solution.ipynb`: solución de referencia explicada.

📝 Evaluación

Criterio	Peso
Comprensión conceptual	25 %
Ejecución reproducible	25 %
Interpretación basada en evidencia	25 %
Riesgos, límites y mejora propuesta	25 %

Consulta assessment.md para preguntas y criterio de aceptación.

⚠️ Errores comunes

Síntoma	Causa probable	Corrección
El código corre, pero no hay conclusión	Se confundió ejecución con aprendizaje	Explica qué demuestra y qué no demuestra
El resultado cambia sin explicación	No se registró semilla o configuración	Conserva semilla, versión y parámetros
Se promete uso real	Se extrapoló desde una demo educativa	Declara entorno, datos, límites y revisión humana
Se copia una métrica aislada	No existe baseline ni costo de error	Añade comparación y criterio de decisión

❓ Preguntas frecuentes

¿Debo usar una API comercial?
No. El núcleo funciona localmente. Las extensiones LIVE se documentan por separado.

¿El laboratorio representa una implementación industrial?
No por sí solo. Enseña el contrato y el patrón; producción exige integración, seguridad, observabilidad, pruebas y operación.

¿Dónde profundizo?
Revisa las especializaciones enlazadas en el README raíz y la ruta siguiente.

🔗 Referencias

- Comisión Europea (2026), *Code of Practice on marking and labelling of AI-generated content* — uso: fuente regulatoria primaria para etiquetas, EU Icons y obligaciones de transparencia
- C2PA. *Content Credentials: C2PA Technical Specification 2.2*. c2pa.org/specifications/specifications/2.2/index.html — uso: marco normativo de referencia
- Kirchenbauer, J. et al. (2023). *A Watermark for Large Language Models*. ICML 2023. arXiv:2301.10226 — uso: fuente primaria del mecanismo estudiado
- Dathathri, S. et al. (2024). *Scalable watermarking for identifying large language model outputs* (SynthID-Text). Nature, 634. doi:10.1038/s41586-024-08025-4 — uso: fuente primaria del mecanismo estudiado
- Content Credentials (implementación de C2PA para usuarios finales): contentcredentials.org — uso: referencia consultada en su fuente original

---

📜 Papers que fundamentan esta clase

> Bloque generado por `python scripts/link_papers_to_classes.py`. La fuente es `papers/catalog/papers.json`.

Paper	Año	Qué desbloqueó	Miniatura
P131 · Una marca de agua para modelos de lenguaje grandes	2023	Deja una firma estadística verificable en el texto generado sesgando qué tokens se eligen, sin degradar apreciablemente la calidad ni necesitar el modelo para detectarla.	notebook

Cada ficha explica el problema anterior, la matemática mínima, los límites y los errores de atribución más frecuentes. Para leerlas con método: cómo leer un paper de IA · anexos matemáticos.

📚 Bibliografía de apoyo

> Bloque generado por `python scripts/link_sources_to_classes.py`. Cada obra lleva su localizador verificado en `sources/bibliography.json`.

Los papers dicen **de dónde salió** el mecanismo. Estas obras lo **desarrollan** con el espacio que una clase no tiene: teoría completa, demostraciones y ejercicios.

Obra	Edición	Localizador	Papel en esta clase
Goodfellow, Ian, Bengio, Yoshua y Courville, Aaron — *Deep Learning*	2016	ISBN 9780262035613 · web de la obra	obra de referencia de la parte 07 · capítulos de modelos generativos

Normas y documentación oficial que aplica esta clase: Code of Practice on marking and labelling of AI-generated content · c2pa.org/specifications

📝 Evaluación completa

❓ Preguntas

1. Define procedencia, marcas y autenticidad sin usar una marca o framework como definición.
2. Explica la relación entre C2PA, watermark, provenance, autenticidad.
3. Ejecuta `lab.py` dos veces con la misma semilla. ¿Qué debe conservarse?
4. Identifica una afirmación permitida y una afirmación exagerada sobre el resultado.
5. Propón una prueba negativa o un caso límite.

🏆 Reto verificable

Amplía el resultado del laboratorio con una clave `student_extension` que incluya:

- el supuesto que estás probando;
- una medición o comprobación;
- la conclusión;
- una limitación.

✅ Criterio de aceptación

- [ ] `lab.py` termina con código 0.
- [ ] El resultado contiene `kind`, `seed`, `evidence` y `limitations`.
- [ ] La extensión no modifica el comportamiento de otras clases.
- [ ] La interpretación referencia datos impresos por el laboratorio.
- [ ] Se declara al menos un riesgo o condición de no uso.

---

099 — Proyecto: pipeline creativo trazable

Parte: 07 — IA generativa para texto, imagen, audio, video y 3D
Nivel: avanzado · **Horas estimadas:** 10
Laboratorio: `capstone` · **Estado:** `EXECUTABLE_CORE`

🎯 Propósito

Comprender **proyecto: pipeline creativo trazable** dentro de la evolución de la inteligencia artificial, implementar un experimento mínimo verificable y distinguir qué parte constituye evidencia frente a una afirmación todavía no comprobada.

📚 Resultados de aprendizaje

Al finalizar podrás:

1. Explicar proyecto: pipeline creativo trazable usando los conceptos `multimedia`, `provenance`, `evaluación`, `publicación`.
2. Ejecutar el laboratorio con una semilla explícita y revisar su contrato JSON.
3. Identificar al menos un supuesto, una limitación y un riesgo de aplicación.
4. Comparar el enfoque con la etapa anterior de la ruta de aprendizaje.
5. Producir una evidencia reproducible y una conclusión que no exceda los datos.

🧩 Conceptos centrales

`multimedia`, `provenance`, `evaluación`, `publicación`

🗺️ Ubicación en el mapa de la IA

Este proyecto integra la parte 07 completa: la generación multimedia (clases 088–096) aporta las etapas creativas, la clase 097 aporta la disciplina sobre datos sintéticos y la 095 aporta procedencia y marcas. El resultado —un pipeline donde cada activo publicado puede responder "quién, con qué modelo, con qué semilla y a partir de qué"— es el patrón que exigen los marcos de gobernanza (NIST AI RMF, C2PA) y el que reutilizarás en las partes siguientes para pipelines de recuperación y agentes.

📖 Fundamentos

🧱 Diseño por etapas con contratos JSON

Un pipeline creativo trazable es una secuencia de etapas donde cada una consume y produce un **contrato JSON explícito**. Contrato mínimo por etapa:

```
{
  "stage": "texto|imagen|edición|...",
  "model": "nombre",  "version": "x.y",
  "seed": 42,
  "prompt_o_params": "...",
  "input_hash":  "hash de la salida de la etapa anterior",
  "output_hash": "hash de la salida propia",
  "timestamp": "ISO-8601",
  "licencia_y_consentimiento": {"fuente": "...", "licencia": "..."}
}
```

Tres decisiones de diseño sostienen la trazabilidad:

1. **Determinismo declarado:** modelo + versión + semilla + parámetros registrados. No garantiza reproducción bit a bit entre hardwares, pero sí auditabilidad: se sabe exactamente qué se pidió y con qué configuración.
2. **Hashes encadenados:** el `input_hash` de la etapa n debe ser igual al `output_hash` de la etapa n−1. Así, el manifiesto final encadena todo el linaje, al estilo de la cadena de manifiestos C2PA.
3. **Registro append-only:** los manifiestos de etapa no se editan; una re-ejecución crea entradas nuevas. Corregir borrando historia destruye la evidencia.

📋 Registro de procedencia por etapa

Cada etapa registra: **modelo y versión** (¿qué generador?), **semilla** (¿qué muestra del espacio de salidas?), **prompt/parámetros** (¿qué se pidió?), **hash de entrada y de salida** (¿sobre qué bytes exactos?). Con eso, el manifiesto final es verificable con el mismo algoritmo de la clase 098: recalcular hashes y recorrer la cadena. Si el pipeline usa datos sintéticos como insumo (clase 097), el manifiesto debe declararlo: fracción sintética, generador de origen y protocolo de utilidad aplicado (TSTR), para no contaminar silenciosamente entrenamientos futuros.

📐 Evaluación y gobernanza

- **Utilidad:** ¿el activo cumple la especificación creativa? Métricas automáticas cuando existan (p. ej. similitud prompt–imagen) + revisión humana registrada.
- **Autenticidad verificable:** la cadena de hashes cierra y, si se publica, el manifiesto viaja con el activo (C2PA / Content Credentials).
- **Gobernanza:** consentimiento y licencia de cada insumo (¿el material de la etapa 1 permitía uso derivado?), atribución, y un mapa de riesgos al estilo NIST AI RMF: identificar (¿qué puede salir mal?), medir (¿con qué métrica?), gestionar (¿quién aprueba la publicación?).

🧮 Ejemplo trabajado

Pipeline de 3 etapas: **texto → imagen → edición**. Usamos un hash simulado de 8 bits (didáctico, no criptográfico): `h(s) = suma de los códigos de los caracteres mod 256`, en hexadecimal.

```
Etapa 1 (texto):    salida  s1 = "un faro al amanecer"
                    h(s1) = 0xF4            → output_hash = F4

Etapa 2 (imagen):   input_hash = F4  ✓ (coincide con la etapa 1)
                    salida  s2 = "IMG[faro,amanecer,seed=7]"
                    h(s2) = 0xE6            → output_hash = E6

Etapa 3 (edición):  input_hash = E6  ✓
                    salida  s3 = "IMG[faro,amanecer,seed=7]+recorte"
                    h(s3) = 0x05            → output_hash = 05

Manifiesto final:   cadena F4 → E6 → 05   → VERIFICA
```

Ahora alguien "retoca" la etapa 2 sin registrarlo: regenera con `seed=8`. La nueva salida s2' produce `h(s2') = 0xE7 ≠ E6`. Al verificar:

```
Etapa 3 declara input_hash = E6, pero h(s2') = E7  → CADENA ROTA en 2→3
```

La verificación no dice *qué* cambió ni *por qué* —solo que los bytes que la etapa 3 declaró consumir ya no existen. Ese es exactamente el comportamiento del hard binding C2PA: detecta la alteración, no la repara. Un hash real (SHA-256) hace además computacionalmente inviable fabricar una s2' con el mismo hash.

📊 Propiedades y comparación

Enfoque de pipeline	Reproducibilidad	Detección de alteraciones	Costo operativo	Riesgo principal
Ad hoc (sin registro)	ninguna	ninguna	cero	imposible auditar o corregir
Log de texto libre	baja (ambigua)	ninguna	bajo	el log se edita sin dejar rastro
Contratos JSON + hashes encadenados	alta (auditable)	sí (cadena rota)	medio	disciplina de registro por etapa
C2PA firmado end-to-end	alta + no repudio	sí, con garantía criptográfica	alto (PKI)	gestión de certificados

⚠️ Errores conceptuales frecuentes

1. **"Registrar la semilla garantiza reproducir el mismo activo."** La semilla fija la muestra *dado* modelo, versión, hardware y librerías; un cambio de versión del modelo produce otra salida con la misma semilla. Por eso el contrato registra ambos.
2. **"La cadena de hashes protege el contenido."** Solo lo *vincula*: detecta alteraciones a posteriori. La protección (quién puede escribir en el registro) es un problema de control de acceso, no de hashing.
3. **"Trazabilidad = burocracia que se añade al final."** Un manifiesto reconstruido retrospectivamente no es evidencia: la procedencia solo vale si se registra en el momento de la generación, dentro del pipeline.
4. **"Si cada etapa es de un proveedor confiable, el pipeline es confiable."** La confianza no compone automáticamente: el eslabón sin registrar (una edición manual entre etapas) rompe el linaje aunque todas las etapas firmadas sean honestas.
5. **"El proyecto es sobre generar contenido bonito."** El entregable evaluable es el *manifiesto verificable*; la calidad creativa sin trazabilidad no aprueba la parte de gobernanza.

🚀 Del aprendizaje a la operación

Para llevar este patrón a producción faltan: hashes criptográficos reales (SHA-256) y firmas con una PKI gestionada en lugar del hash didáctico de 8 bits; un almacén de manifiestos append-only con control de acceso; integración C2PA real en los formatos de salida (JPEG, MP4) con Content Credentials; revisión legal de licencias y consentimiento por jurisdicción; y un proceso de aprobación humana previo a la publicación alineado con NIST AI RMF (govern–map–measure–manage).

🧪 Laboratorio

```
python lab.py
```

El laboratorio llama a `ai_evolution.labs.run_lab("capstone")`. Esta decisión evita una implementación divergente por clase: cada una conserva un entrypoint propio, pero los motores didácticos se prueban como una biblioteca común.

🔍 Evidencia esperada

- tipo de laboratorio y semilla;
- entradas o decisiones observables;
- resultado estructurado;
- lista `evidence` con hechos que pueden inspeccionarse;
- lista `limitations` que impide presentar la demo como producción.

📓 Notebooks

- 📓 `notebook.ipynb`: recorrido guiado con la materia resumida.
- ✍️ `notebook_student.ipynb`: ejercicios para resolver.
- ✅ `notebook_solution.ipynb`: solución de referencia explicada.

📝 Evaluación

Criterio	Peso
Comprensión conceptual	25 %
Ejecución reproducible	25 %
Interpretación basada en evidencia	25 %
Riesgos, límites y mejora propuesta	25 %

Consulta assessment.md para preguntas y criterio de aceptación.

⚠️ Errores comunes

Síntoma	Causa probable	Corrección
El código corre, pero no hay conclusión	Se confundió ejecución con aprendizaje	Explica qué demuestra y qué no demuestra
El resultado cambia sin explicación	No se registró semilla o configuración	Conserva semilla, versión y parámetros
Se promete uso real	Se extrapoló desde una demo educativa	Declara entorno, datos, límites y revisión humana
Se copia una métrica aislada	No existe baseline ni costo de error	Añade comparación y criterio de decisión

❓ Preguntas frecuentes

¿Debo usar una API comercial?
No. El núcleo funciona localmente. Las extensiones LIVE se documentan por separado.

¿El laboratorio representa una implementación industrial?
No por sí solo. Enseña el contrato y el patrón; producción exige integración, seguridad, observabilidad, pruebas y operación.

¿Dónde profundizo?
Revisa las especializaciones enlazadas en el README raíz y la ruta siguiente.

🔗 Referencias

- C2PA. *Content Credentials: C2PA Technical Specification 2.2*. c2pa.org/specifications/specifications/2.2/index.html — uso: marco normativo de referencia
- NIST. *AI Risk Management Framework (AI RMF 1.0)*. nist.gov/itl/ai-risk-management-framework — uso: marco normativo de referencia
- Ho, J., Jain, A. y Abbeel, P. (2020). *Denoising Diffusion Probabilistic Models*. arXiv:2006.11239 — uso: fuente primaria del mecanismo estudiado
- Rombach, R. et al. (2022). *High-Resolution Image Synthesis with Latent Diffusion Models*. arXiv:2112.10752 — uso: fuente primaria del mecanismo estudiado
- Content Credentials: contentcredentials.org — uso: referencia consultada en su fuente original

📚 Bibliografía de apoyo

> Bloque generado por `python scripts/link_sources_to_classes.py`. Cada obra lleva su localizador verificado en `sources/bibliography.json`.

Los papers dicen **de dónde salió** el mecanismo. Estas obras lo **desarrollan** con el espacio que una clase no tiene: teoría completa, demostraciones y ejercicios.

Obra	Edición	Localizador	Papel en esta clase
Goodfellow, Ian, Bengio, Yoshua y Courville, Aaron — *Deep Learning*	2016	ISBN 9780262035613 · web de la obra	obra de referencia de la parte 07 · capítulos de modelos generativos

Normas y documentación oficial que aplica esta clase: c2pa.org/specifications · AI Risk Management Framework

📝 Evaluación completa

❓ Preguntas

1. Define proyecto: pipeline creativo trazable sin usar una marca o framework como definición.
2. Explica la relación entre multimedia, provenance, evaluación, publicación.
3. Ejecuta `lab.py` dos veces con la misma semilla. ¿Qué debe conservarse?
4. Identifica una afirmación permitida y una afirmación exagerada sobre el resultado.
5. Propón una prueba negativa o un caso límite.

🏆 Reto verificable

Amplía el resultado del laboratorio con una clave `student_extension` que incluya:

- el supuesto que estás probando;
- una medición o comprobación;
- la conclusión;
- una limitación.

✅ Criterio de aceptación

- [ ] `lab.py` termina con código 0.
- [ ] El resultado contiene `kind`, `seed`, `evidence` y `limitations`.
- [ ] La extensión no modifica el comportamiento de otras clases.
- [ ] La interpretación referencia datos impresos por el laboratorio.
- [ ] Se declara al menos un riesgo o condición de no uso.
